CYCLE INGÉNIEUR D'ÉTAT

Rapport De Projet de Fin d'Annèe

Filière : 4 ème Ingénierie des Systèmes Informatiques.

Thème : Smart Confirm

Rédigé(e) et présenté(e) par :

BENAAZZOUZ MOHAMMED

EL KHAYARI AMINE

Encadré par :

M. Y.TABII

Jury :

M. Y.TABII

Année Universitaire 2025/2026

Dédicaces :

Notre projet n'aura pas vu le jour sans ceux qui nous ont aidés par leurs patiences leurs encouragements et nous ont soutenus dans chacun de nos instants de faiblesse,

Nous dédions ce travail

A nos familles qui sont la source de notre motivation et notre espoir

Sans leur extrême attention, leurs sacrifices et leurs encouragements tout au long de nos études, nous ne serons pas ce que nous sommes
devenus aujourd'hui.
J'espère qu'ils puissent trouver dans ce modeste travail un témoignage de respect et d'affection envers eux en espérant que ça
soit à la hauteur de leurs attentes.

A nos amis et nos collègues

Pour leur support et pour tous les bons moments qu'nous avons vécus ensemble.
J'espère que notre amitié durera éternellement.

Remerciement :

Je commence par exprimer ma profonde gratitude envers Dieu, le Tout-Puissant, pour Sa guidance, Sa grâce infinie et Sa bienveillance tout au long de mon parcours académique. Sans Sa lumière et Sa miséricorde, je n'aurais pas pu accomplir cette étape importante de ma vie.

Nous tenons à adresser en premier lieu nos remerciements et nos profondes gratitudes à notre encadrant de Projet de Fin d'étude Pr. **Y.Tabii** pour nous avoir permis de réaliser ce travail riche en nouveauté. Nous le remercions très sincèrement pour son encadrement, son soutien, son aide précieuse et sa confiance sans lesquels ce travail n'aurait pas été ce qu'il est.

Nos remerciements vont également les membres du jury de notre travail d'avoir accepté de juger mon travail.

Nos remerciements aussi à tous nos professeurs qui nous ont enseigné pour leurs efforts fournis tout au long de notre formation.

Nos remercîments vont aussi à tout le corps administratif de ISGA.

Et finalement merci à nos amis (es) et à tous ceux qui ont participé de près ou de loin à l'élaboration de ce travail.

A Nos parents qui ont toujours cru en nous et nous ont soutenus. Aucun hommage ne pourrait être à la hauteur de leur sacrifice et leur soutien inconditionnel tant que matériel et moral.

A tous les membres de notre famille, et à nos amis.

A tous nos professeurs tout au long de notre parcours scolaire.

A tous ceux qui ont contribué de près ou de loin pour que ce projet soit possible.

Résumé :

Dans le cadre de notre formation en 4ème Année du Cycle Ingénieur en Génie Informatique à l'Institut Supérieur d'Ingénierie et des Affaires (ISGA), nous avons conçu et réalisé le projet SmartConfirm, une plateforme SaaS (Software-as-a-Service) multi-tenant innovante dédiée au commerce conversationnel, à l'automatisation e-commerce et à l'intégration d'agents d'intelligence artificielle sur WhatsApp.

Dans les marchés émergents d'Afrique du Nord et du Moyen-Orient (MENA), le commerce électronique repose massivement sur le paiement à la livraison (Cash-On-Delivery - COD). Ce modèle est confronté à une problématique critique : un taux d'échec de livraison et de retour de colis variant entre 20% et 40%, causé par des adresses imprécises, des commandes impulsives et l'absence de vérification préalable. SmartConfirm répond à cette problématique en automatisant instantanément la confirmation des commandes via WhatsApp.

Développée sur la base d'une architecture moderne et modulaire, l'application s'appuie sur NestJS (TypeScript) côté backend, React 18 avec Vite et Tailwind CSS côté frontend, ainsi qu'une base de données PostgreSQL pilotée par TypeORM et des files d'attente gérées par Redis et BullMQ. L'intégration de WhatsApp repose sur le moteur WebSocket Baileys, éliminant les coûts prohibitifs de l'API Cloud officielle. La solution intègre des agents LLM avancés (OpenAI GPT-4o, Google Gemini) couplés à la reconnaissance vocale OpenAI Whisper (STT) et à la synthèse vocale neuronale (TTS). Enfin, un modèle économique SaaS complet est implémenté avec Stripe et PayPal Subscriptions, incluant des périodes d'essai de 3 jours, la gestion stricte des quotas et un workflow sécurisé de demande de remboursement sous 48 heures soumis à l'approbation administrative.

Abstract :

As part of the curriculum for the 4th Year of the Engineering Degree in Computer Engineering at ISGA (Institut Supérieur d'Ingénierie et des Affaires), we developed SmartConfirm, an innovative multi-tenant Software-as-a-Service (SaaS) platform for WhatsApp conversational commerce, e-commerce automation, and AI-driven conversational agents.

In emerging markets across North Africa and the Middle East (MENA), e-commerce heavily relies on Cash-on-Delivery (COD). This model suffers from a significant bottleneck: order return and failure rates ranging from 20% to 40% due to imprecise customer addresses, impulsive purchasing, and lack of immediate confirmation. SmartConfirm tackles this challenge by fully automating real-time order verification, customer support, and post-purchase communication through WhatsApp.

Built upon a scalable and modular clean architecture, the backend is engineered with NestJS (TypeScript), TypeORM with PostgreSQL and Redis/BullMQ message queues, while the frontend leverages React 18, Vite, and Tailwind CSS. WhatsApp connectivity is powered by the Baileys multi-device WebSocket engine, circumventing high recurring Meta Cloud API fees. The platform features hybrid AI conversational agents (OpenAI GPT-4o, Google Gemini) capable of understanding Moroccan Darija, Arabic, and French audio notes via OpenAI Whisper STT and replying with natural voice audio via Neural TTS. A complete monetization engine handles Stripe and PayPal Subscriptions, supporting a 3-day free trial on paid plans, dynamic quota enforcement, and a 48-hour customer refund request workflow monitored by super-administrators.

Table des matières

Résumé :....4

Abstract :....5

Introduction Générale....8

1. Contexte du projet :....8

2. Problématique :....8

3. Objectifs du projet :....9

4. Méthodologie utilisée :....10

Chapitre 1 : Présentation Générale....11

1.1. Présentation de l'école ISGA :....11

1.2. Présentation du projet :....11

1.3. Contexte professionnel et justification du choix....11

Chapitre 2 : Étude et Conception....13

2.1. Analyse des besoins :....13

Acteurs du système :....13

2.2. Diagramme de cas d'utilisation (Use Case) :....14

Cas d'utilisation principaux :....16

2.3. Diagramme de classes UML....17

Principales classes :....17

2.4. Diagramme de Sequence UML....18

2.5. Maquettes et conception de l'interface....19

Chapitre 3 : Réalisation du Projet....20

3.1. Choix techniques....20

Langages & Frameworks :....20

Conception & Maquettes :....23

Hébergement & déploiement :....23

3.2. Architecture technique du projet....24

Structure simplifiée :....25

3.3. Fonctionnalités développées....26

3.4. Captures d'écran de l'interface....27

3.5. Interfaces utilisateur – Partie publique (client)....28

Figure 1 : Formulaire de connexion / inscription utilisateur....28

Figure 2 : Page d'accueil de l'application PayPark....29

Figure 3 : Formulaire d'ajout d'un véhicule....33

Figure 4 : Interface de recherche de parking par localisation.....34
.....34
3.6. Interfaces utilisateur – Partie propriétaire (owner).....38
Figure 10 : Tableau de bord du propriétaire.....38
.....38
Figure 11 : Formulaire d'ajout d'un nouveau parking.....38
.....38
3.7. Interfaces utilisateur – Partie administrateur (admin).....41
.....44
Chapitre 4 : Tests et Déploiement.....45
4.1. Stratégie de test.....45
Tests de validation :.....46
4.2. Résultats des tests.....47
Environnement de développement local :.....48
Déploiement en ligne :.....48
Outils utilisés pour le déploiement :.....48
Chapitre 5 :.....50
Difficultés rencontrées et perspectives d'évolution.....50
5.1. Difficultés rencontrées.....50
Déploiement et conteneurisation cloud :.....50
5.2. Limites du projet.....51
5.3. Perspectives d'évolution.....51
Monétisation et exploitation commerciale :.....51
Intelligence Artificielle & automatisation avancée :.....51
Améliorations techniques & Écosystème :.....52
Améliorations techniques :.....52
Conclusion Générale.....53
Bibliographie / Webographie.....54

Introduction Générale

1. Contexte du projet :

E-commerce connaît une croissance exponentielle à l'échelle internationale, et particulièrement sur le continent africain et dans la zone MENA. Cependant, contrairement aux marchés occidentaux où le paiement en ligne par carte bancaire est largement dominant, l'écosystème commercial régional repose à plus de 80% sur le modèle du paiement à la livraison (Cash-on-Delivery ou COD).

Si ce modèle favorise l'acte d'achat en rassurant les consommateurs, il engendre pour les marchands et les plateformes logistiques des coûts opérationnels majeurs. En moyenne, les marchands e-commerce de la région constatent un taux de retour ou d'échec de livraison alarmant oscillant entre 20% et 40% de leurs commandes globales. Face à cette réalité, l'application WhatsApp s'est imposée comme le canal de communication privilégié et quasi-universel dans le quotidien des consommateurs, avec un taux d'ouverture supérieur à 95% dans les cinq minutes suivant la réception d'un message.

2. Problématique :

Comment concevoir, développer et déployer une plateforme logicielle SaaS multi-tenant capable d'automatiser instantanément la confirmation des commandes e-commerce, d'offrir des interactions d'intelligence artificielle textuelles et vocales avancées sur WhatsApp sans surcoût par message, tout en fournissant une infrastructure économique pérenne intégrant abonnements, gestion de quotas et remboursements sécurisés ?

Cette problématique soulève des questions liées à la centralisation des données, à la gestion multi-acteurs (administrateurs, propriétaires, utilisateurs) et à l'expérience utilisateur dans un environnement numérique.

3. Objectifs du projet :

Objectif général :

Développer une plateforme SaaS complète permettant l'automatisation et la confirmation intelligente des commandes e-commerce via WhatsApp, avec des fonctionnalités adaptées aux différents profils d'utilisateurs.

Objectifs spécifiques :

Concevoir une interface web ergonomique et responsive offrant une expérience utilisateur fluide et intuitive (React 19, Tailwind CSS).

- Permettre aux e-commerçants de connecter leurs boutiques (Shopify, WooCommerce, YouCan) et d'automatiser l'envoi des messages de confirmation et de suivi de commande.
- Intégrer une passerelle WhatsApp multi-sessions simple d'utilisation via l'appairage en direct par QR Code (Baileys, WebSockets).
- Fournir aux administrateurs un tableau de bord de supervision complet pour le suivi analytique, la gestion des boutiques et des volumes de messages (Recharts, TanStack Query).
- Mettre en place un système de gestion d'abonnements et de quotas adapté au modèle commercial SaaS.
- Utiliser des technologies web modernes et robustes : React, NestJS, TypeScript, PostgreSQL, TypeORM, Redis, Docker, GitHub...
- Assurer la sécurité des données, la persistance chiffrée des sessions et la haute disponibilité du système.

4. Méthodologie utilisée :

La réalisation de ce projet s'est basée sur une approche agile, avec des étapes progressives :

- **Analyse des besoins des utilisateurs** à travers la rédaction d'un cahier des charges fonctionnel et technique centré sur les flux e-commerce.
- **Modélisation UML** pour représenter l'architecture globale, les cas d'utilisation, les diagrammes de séquence et la structure des données.
- **Conception de l'interface utilisateur (UI/UX)** afin de garantir un tableau de bord moderne, réactif et intuitif.
- **Développement de la plateforme** en utilisant **React 19** côté frontend et **NestJS (TypeScript)** côté backend, couplés à **PostgreSQL**, **Redis/BullMQ** et **Baileys**, avec gestion du code source via **Git et GitHub**.
- **Tests unitaires et fonctionnels** (Jest, E2E), conteneurisation complète avec **Docker** et déploiement de l'infrastructure sur serveur cloud.

Chapitre 1 : Présentation Générale

1.1. Présentation de l'école ISGA :

Fondé en 1981, l'Institut Supérieur d'Ingénierie et des Affaires (ISGA) est un établissement d'enseignement supérieur d'excellence reconnu par l'État marocain. Avec plus de quatre décennies d'expertise pédagogique et un réseau de campus modernes répartis dans les principales métropoles du Royaume (Casablanca, Rabat, Marrakech, Fès), l'ISGA forme des ingénieurs d'État et des managers hautement qualifiés.

Au sein de l'École d'Ingénieurs de l'ISGA, la filière Génie Informatique dispense une formation d'avant-garde axée sur le génie logiciel, les architectures applicatives distribuées, le Cloud Computing, la cybersécurité et l'Intelligence Artificielle.

1.2. Présentation du projet :

SmartConfirm est une plateforme logicielle SaaS multi-tenant conçue pour révolutionner la relation client entre les commerçants en ligne et leurs acheteurs via WhatsApp. Elle centralise les flux transactionnels issus des sites e-commerce et automatise la conversation avec l'utilisateur final.

1.3. Contexte professionnel et justification du choix

Dans un monde où la technologie transforme progressivement le commerce électronique, la confirmation des commandes reste encore largement manuelle ou peu optimisée. C'est lors de notre participation avec un collègue à un événement dédié au e-commerce à Casablanca que cette réalité s'est imposée à nous : en observant les stands et en échangeant avec les marchands, nous avons constaté qu'ils travaillaient tous avec

des méthodes classiques, sans aucune automatisation pour valider et sécuriser leurs commandes.

Face à ce constat de terrain, l'idée de développer une solution digitale et automatisée a émergé comme une réponse logique et pertinente.

Ce projet a été choisi pour son utilité concrète pour les commerçants, sa réalisation technique complète (alliant l'automatisation WhatsApp, une architecture SaaS multi-boutiques et la gestion des flux en temps réel), et sa capacité à évoluer vers des fonctionnalités plus avancées, telles que l'assistance conversationnelle par intelligence artificielle ou la qualification automatique des messages vocaux.

ÉLÉMENT	DÉTAIL
OBJECTIF PRINCIPAL	Créer une plateforme SaaS pour l'automatisation et la confirmation intelligente des commandes e-commerce via WhatsApp.
UTILISATEURS CIBLES	Administrateurs (Superadmin), e-commerçants (gérants de boutiques), clients finaux (acheteurs).
FONCTIONNALITÉS ATTENDUES	Appairage WhatsApp (QR Code), intégration multi-boutiques (Shopify, WooCommerce, YouCan), envoi automatisé de messages, gestion des plans & quotas, tableau de bord analytique.
CONTRAINTES TECHNIQUES	Frontend en React 19 (TypeScript, Vite), backend NestJS, passerelle WhatsApp (Baileys), base de données PostgreSQL, files d'attente Redis/BullMQ, architecture conteneurisée (Docker).
LIVRABLES	Plateforme SaaS déployée et fonctionnelle + code source sur GitHub + documentation d'API (Swagger) + rapport de projet de fin d'études.

Chapitre 2 : Étude et Conception

2.1. Analyse des besoins :

Le projet **SmartConfirm** vise à fournir une solution SaaS robuste, intuitive et automatisée pour la confirmation et le suivi des commandes e-commerce via WhatsApp. L'analyse fonctionnelle repose sur l'identification des différents profils d'utilisateurs et de leurs besoins spécifiques :

Acteurs du système :

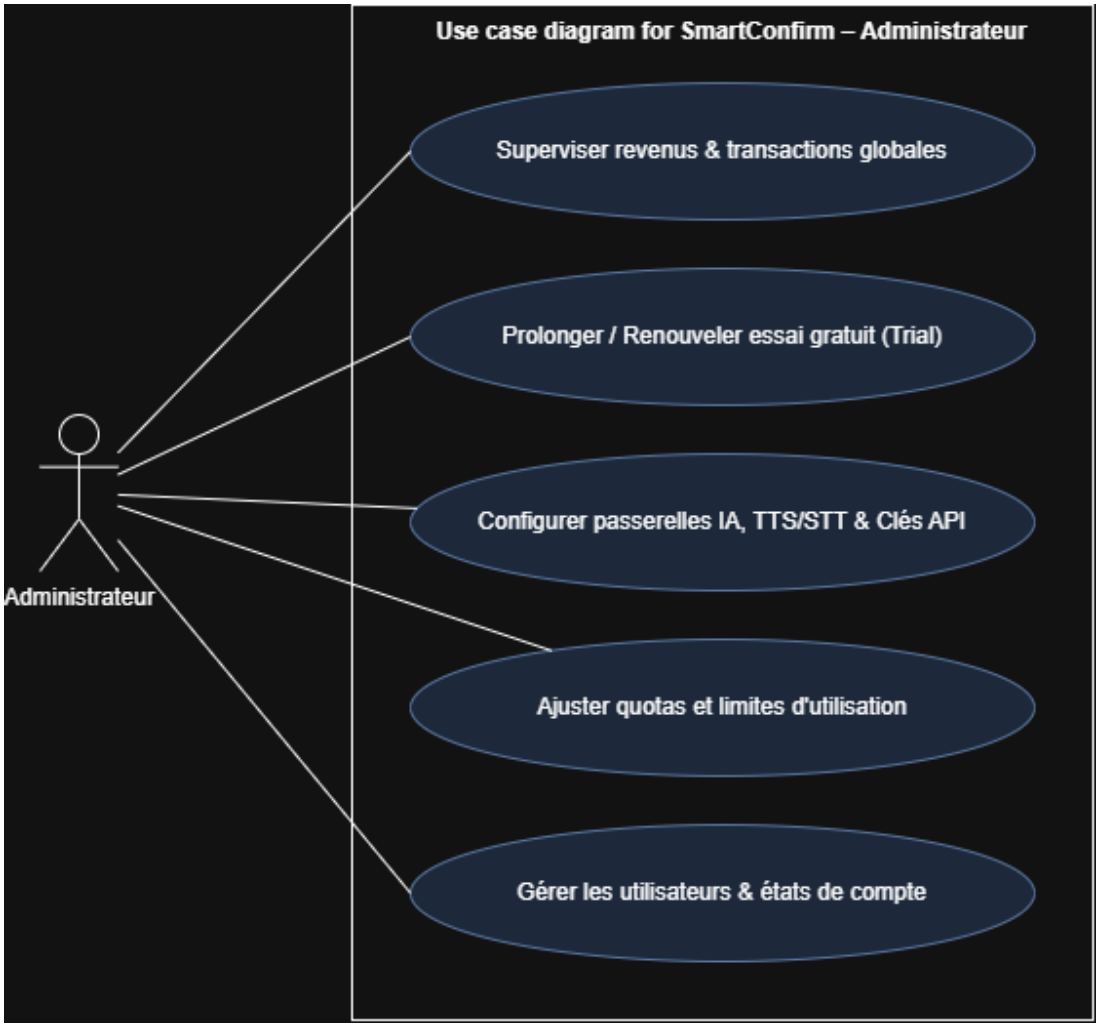
- **Administrateur** :
    - **Supervision globale :** disposer d'un tableau de bord centralisé pour visualiser les métriques clés de la plateforme (revenus, marchands actifs, volumes de messages, état des instances).
    - **Gestion des utilisateurs et boutiques :** gérer les comptes marchands (activation, suspension) et auditer les boutiques raccordées.
    - **Gestion des offres SaaS :** configurer les plans d'abonnement, fixer les quotas d'usage, accorder des périodes d'essai (*free trials*) et traiter d'éventuelles demandes de remboursement.
    - **Surveillance technique :** surveiller la santé des passerelles WhatsApp, la charge des files d'attente Redis/BullMQ et les journaux système.
- **Merchant** :
    - **Connexion et intégration de boutiques :** connecter facilement une ou plusieurs boutiques en ligne (Shopify, WooCommerce, YouCan) via des clés API ou des Webhooks sécurisés.
    - **Gestion des sessions WhatsApp :** appairer un compte WhatsApp professionnel par simple scan d'un QR Code en direct et surveiller l'état de la connexion.
    - **Automatisation des messages :** configurer et personnaliser des modèles de messages pour la confirmation instantanée de commande (Cash on Delivery), le suivi de livraison ou la relance des paniers abandonnés.
    - **Tableau de bord et historique :** suivre en temps réel le statut d'envoi et de délivrabilité des messages ainsi que les taux de confirmation.

- **Gestion de l'abonnement :** choisir un plan tarifaire, suivre sa consommation de quotas (nombre de sessions, volume de messages autorisés) et gérer sa facturation.

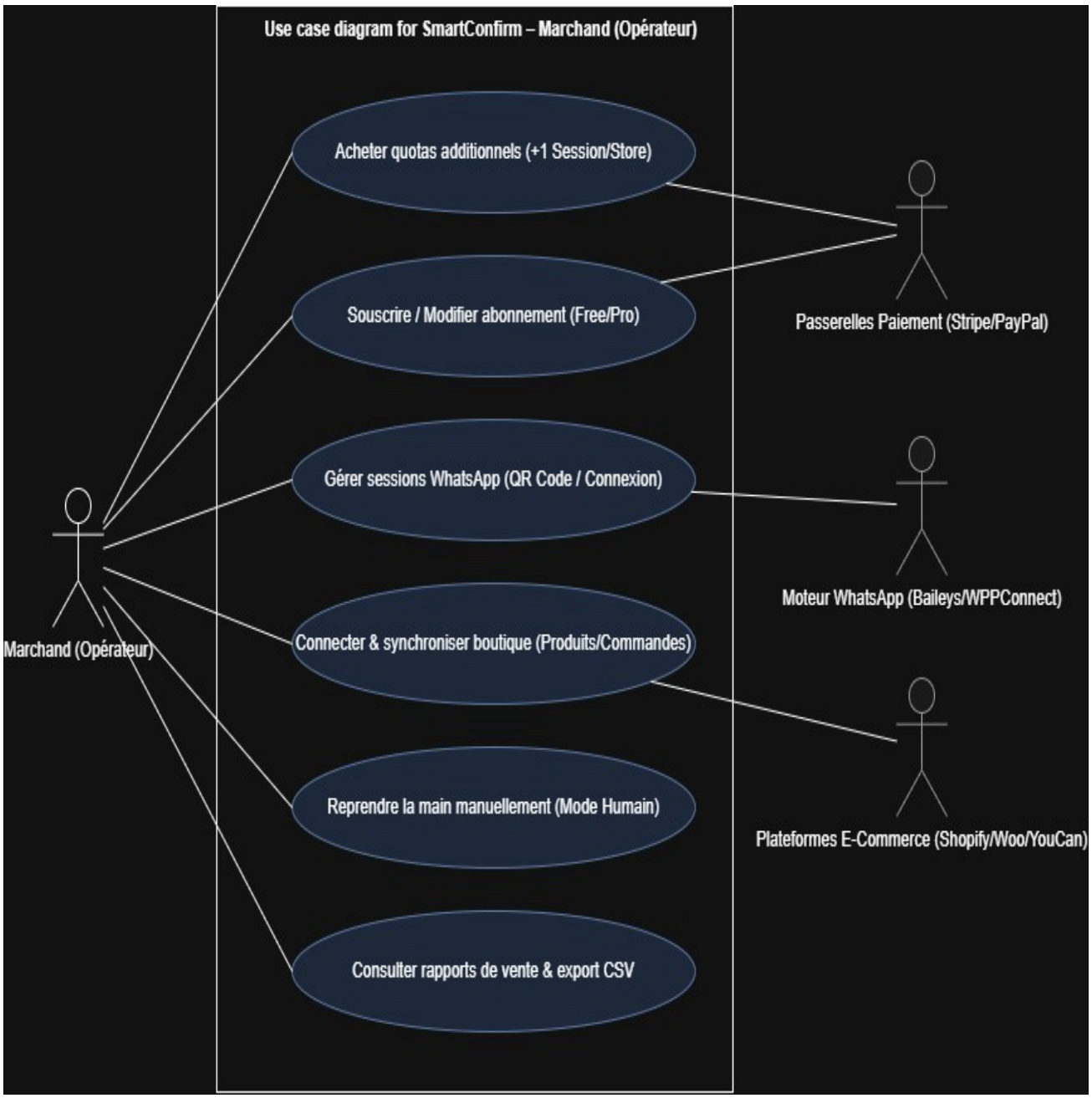
- **Client (Utilisateur)** :
  - **Réception instantanée :** recevoir une notification claire sur WhatsApp dès la validation d'une commande.
  - **Interaction simplifiée :** confirmer, modifier ou annuler une commande facilement par une simple réponse ou via des boutons d'action interactifs.
  - **Suivi de commande :** être informé en temps réel des étapes de livraison de leur colis sans avoir à installer d'application supplémentaire.

2.2. Diagramme de cas d'utilisation (Use Case) :

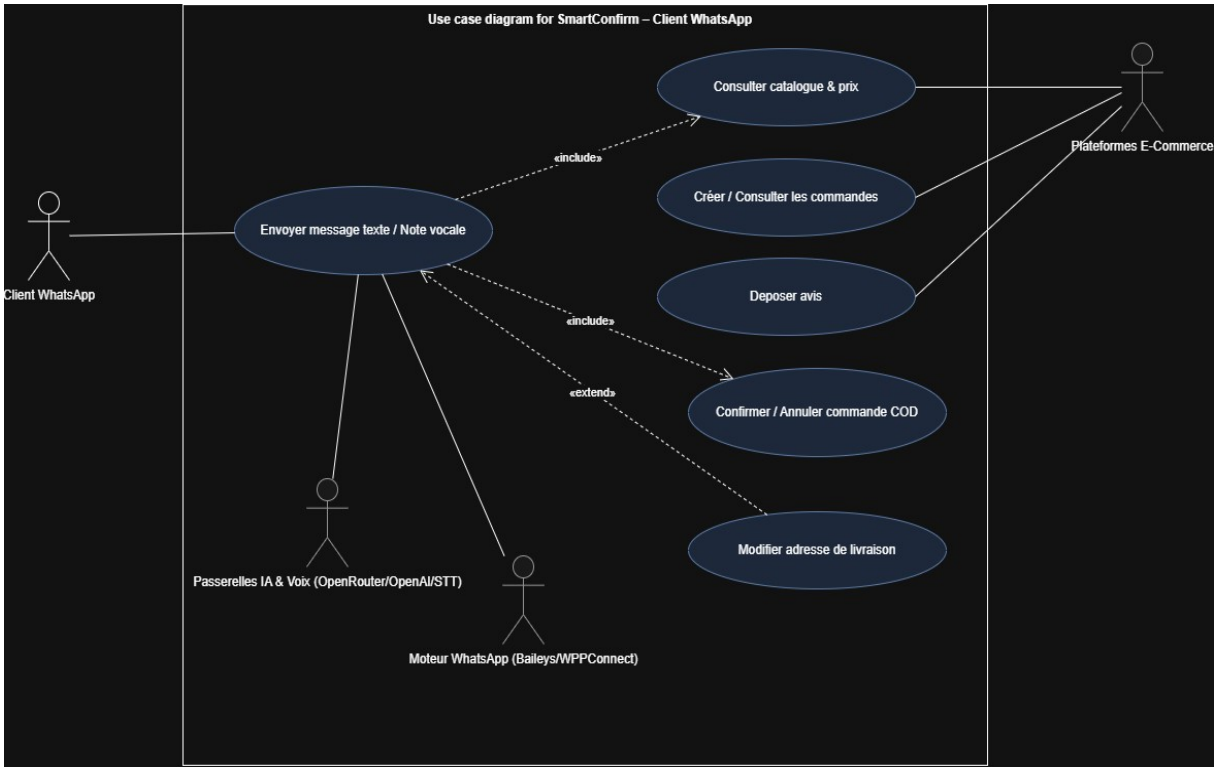
→ **Administrateur :**



→ **Merchant :**


Use case diagram for SmartConfirm – Marchand (Opérateur)
Acheter quotas additionnels (+1 Session/Store)
Souscrire / Modifier abonnement (Free/Pro)
Gérer sessions WhatsApp (QR Code / Connexion)
Connecter & synchroniser boutique (Produits/Commandes)
Reprendre la main manuellement (Mode Humain)
Consulter rapports de vente & export CSV
Marchand (Opérateur)
Passerelles Paiement (Stripe/PayPal)
Moteur WhatsApp (Baileys/WPPConnect)
Plateformes E-Commerce (Shopify/Woo/YouCan)

→ **Client :**



Cas d'utilisation principaux :

- **Le client (WhatsApp) peut :** recevoir la notification de commande, confirmer ou annuler un achat COD, modifier son adresse de livraison, consulter le catalogue, poser des questions par texte/audio (IA) et déposer un avis après réception du colis.

- **L'e-commerçant (Marchand) peut :** créer un compte, connecter sa session WhatsApp via QR Code, intégrer ses boutiques (Shopify, WooCommerce, YouCan), automatiser les messages de confirmation, reprendre la main via le Live Chat et gérer son abonnement SaaS.

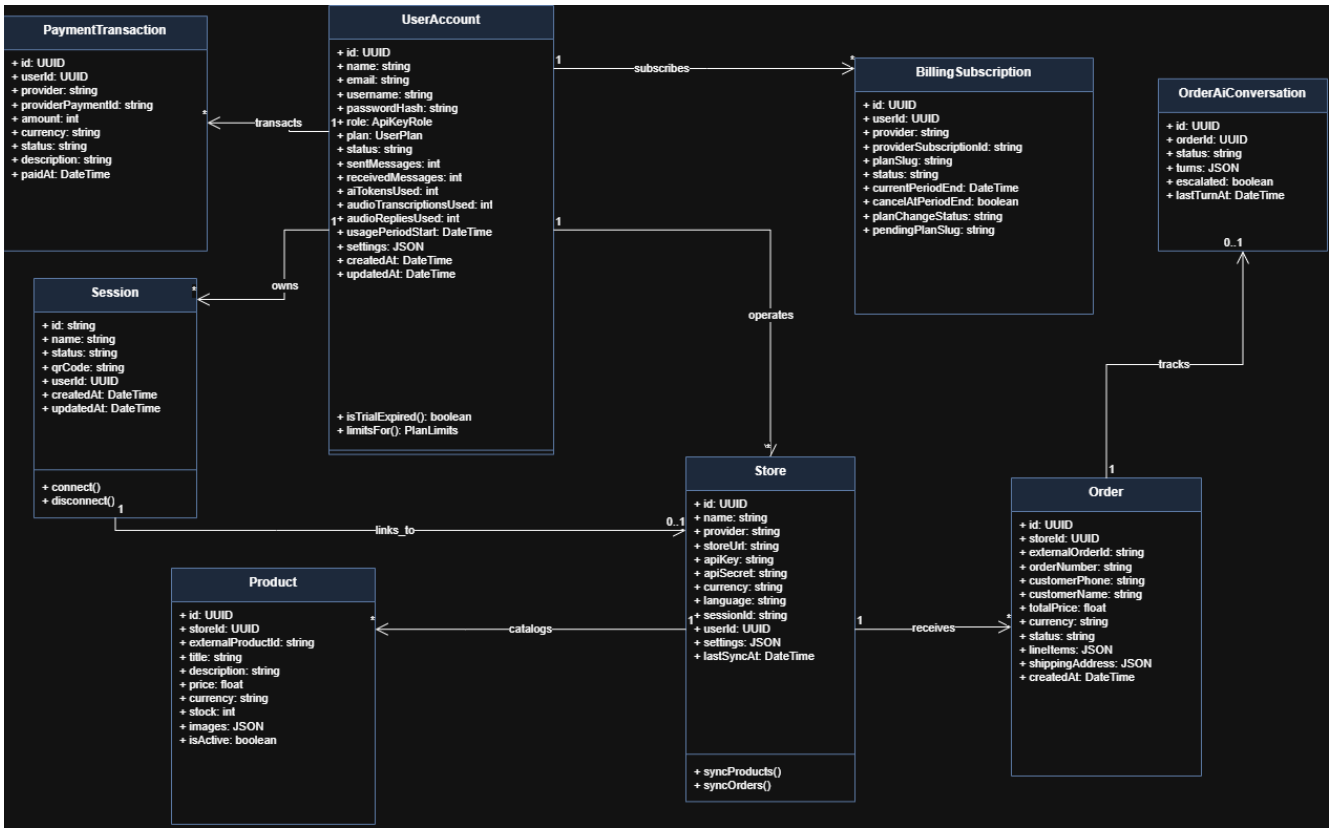
- **L'administrateur peut :** gérer l'ensemble des marchands et boutiques, bloquer ou activer un compte, configurer les forfaits SaaS et quotas, traiter les demandes de remboursement et consulter les statistiques globales.

2.3. Diagramme de classes UML

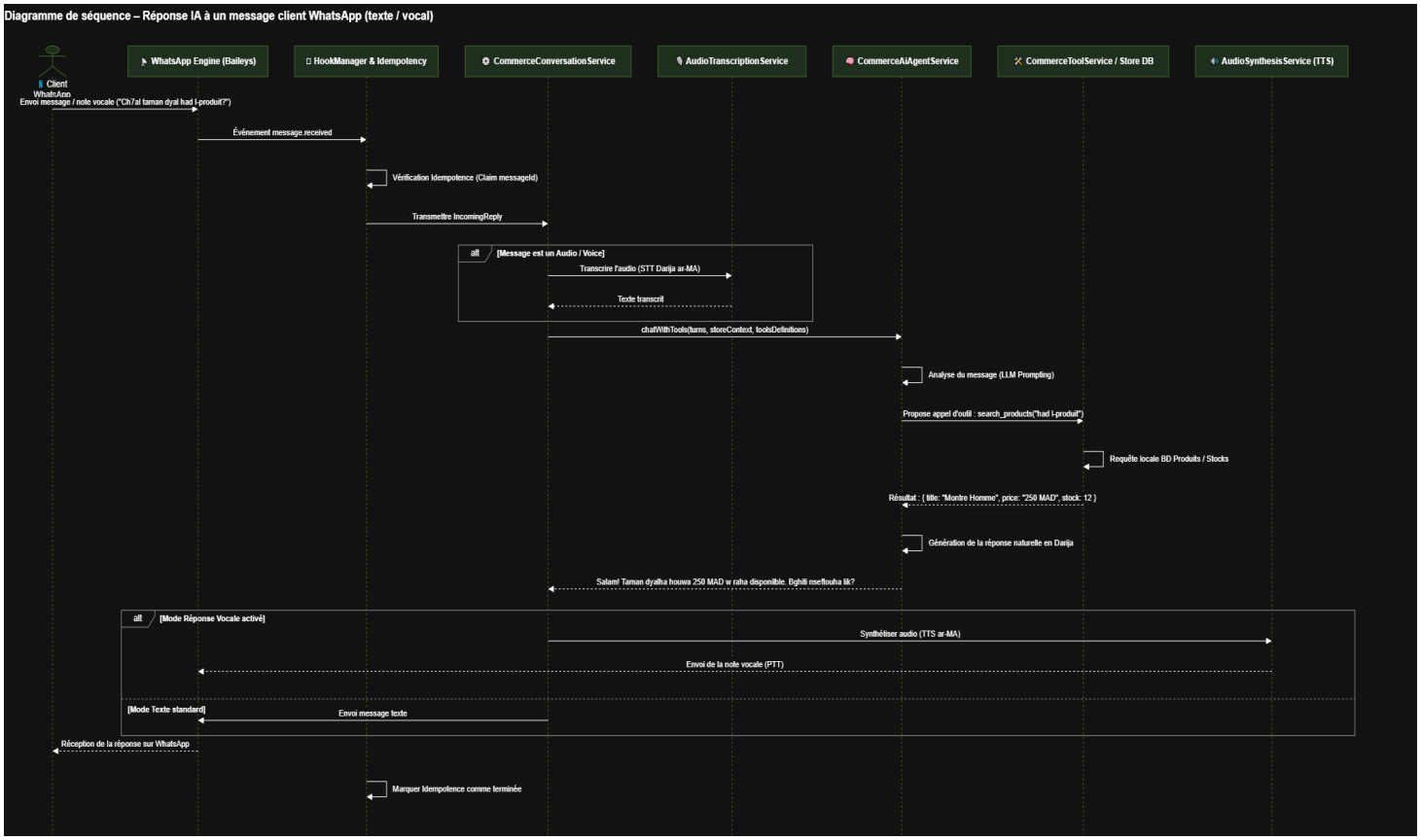
Un diagramme de classes permet de visualiser les entités du système et leurs relations :

Principales classes :

- **UserAccount** (id, name, email, username, passwordHash, role, plan, status, sentMessages, receivedMessages, aiTokensUsed, audioTranscriptionsUsed, audioRepliesUsed, usagePeriodStart, settings, createdAt, updatedAt)
- **Session** (id, name, status, qrCode, userId, createdAt, updatedAt)
- **Store** (id, name, provider, storeUrl, apiKey, apiSecret, currency, language, sessionId, userId, settings, lastSyncAt)
- **Product** (id, storeId, externalProductId, title, description, price, currency, stock, images, isActive)
- **Order** (id, storeId, externalOrderId, orderNumber, customerPhone, customerName, totalPrice, currency, status, lineItems, shippingAddress, createdAt)
- **OrderAiConversation** (id, orderId, status, turns, escalated, lastTurnAt)
- **BillingSubscription** (id, userId, provider, providerSubscriptionId, planSlug, status, currentPeriodEnd, cancelAtPeriodEnd, planChangeStatus, pendingPlanSlug)
- **PaymentTransaction** (id, userId, provider, providerPaymentId, amount, currency, status, description, paidAt)



2.4. Diagramme de Sequence UML



2.5. Maquettes et conception de l'interface

Avant le développement, des **maquettes UI** ont été créées à l'aide de **Photoshop** et **Balsamiq** pour définir le design général de l'application.

Pages conçues :

- Page d'accueil (présentation)
- Espace de connexion / inscription
- Tableau de bord admin
- Espace propriétaire avec gestion des parkings
- Page de recherche et réservation côté client

Ces maquettes ont servi de base pour le développement en HTML/CSS et ont été pensées pour être **responsive** et **ergonomiques**, afin de s'adapter à tous les types d'écrans (ordinateurs, tablettes, smartphones).

Chapitre 3 : Réalisation du Projet

3.1. Choix techniques

Le développement de l'application **SmartConfirm** s'est basé sur un ensemble de technologies web modernes, choisies pour leur efficacité, leur souplesse et leur intégration facile :

Langages & Frameworks :

NestJS (Node.js) a été choisi pour son architecture modulaire en couches (*Clean Architecture*), sa gestion sécurisée des routes, des formulaires (DTO) et de l'authentification par tokens JWT. Côté frontend, React 19 avec TypeScript et Vite permet de concevoir une Single Page Application (SPA) ultra-rapide, fluide et réactive, stylisée de façon moderne via Tailwind CSS.

TECHNOLOGIE	DESCRIPTION
nest	Framework backend sous Node.js offrant une architecture modulaire robuste, l'injection de dépendances et la gestion native des WebSockets.
React	Bibliothèque frontend basée sur des composants réutilisables et un *Virtual DOM* assurant un affichage dynamique et performant sans rechargement de page.

TECHNOLOGIE	DESCRIPTION
TypeScript	Langage typé de Microsoft utilisé sur tout le projet pour fiabiliser le code, réduire les bugs à l'exécution et faciliter la maintenance.
tailwindcss	Framework CSS utilitaire permettant de concevoir rapidement une interface utilisateur épurée, moderne et totalement adaptative (*responsive*).

Base de données :

TECHNOLOGIE	DESCRIPTION
PostgreSQL	SGBD relationnel open-source puissant, réputé pour sa robustesse, sa stabilité et ses performances sur les volumes de données élevés.
TypeORM	ORM pour TypeScript facilitant la modélisation objet des entités et automatisant les migrations de schéma de base de données.
redis	Base NoSQL clé-valeur en mémoire (*In-Memory*), utilisée pour le cache haute vitesse et le pilotage des files d'attente asynchrones.

Outils de développement :

OUTIL	DESCRIPTION
	Framework backend sous Node.js offrant une architecture modulaire robuste, l'injection de dépendances et la gestion native des WebSockets.
	Système de versionnement utilisé pour suivre l'évolution du code source.
	Plateforme collaborative permettant d'héberger le code source, collaborer et suivre les versions.
	Éditeur de code léger et puissant, utilisé pour écrire, structurer et déboguer le code.
	Outil de modélisation UML utilisé pour concevoir les diagrammes (cas d'utilisation, classes...).

Conception & Maquettes :

TECHNOLOGIE	DESCRIPTION
Adobe Photoshop	Utilisé pour réaliser des wireframes (maquettes basse fidélité), notamment lors de la phase de planification.
balsamiq	Utilisé pour réaliser des wireframes (maquettes basse fidélité), notamment lors de la phase de planification.

Paiement en ligne :

TECHNOLOGIE	DESCRIPTION
PayPal	Intégré également pour permettre aux utilisateurs de réserver une place et payer directement via leur compte PayPal. Fonctionne en mode test pour le moment.
stripe	Solution de paiement moderne intégrée en mode test. Permet le traitement des paiements en ligne par carte bancaire.

Hébergement & déploiement :

TECHNOLOGIE	DESCRIPTION
docker	Solution de conteneurisation permettant d'isoler l'API, la base de données et Redis dans des environnements standardisés et reproductibles.
Railway	Plateforme cloud moderne (PaaS) utilisée pour héberger l'application, exécuter le conteneur Docker et gérer les variables d'environnement de production.

3.2. Architecture technique du projet

L'application repose sur une **architecture modulaire en couches (***Clean Architecture***)** couplée à un modèle client-serveur (SPA / API REST), propre à **NestJS** et **React**, qui permet de bien séparer les responsabilités :

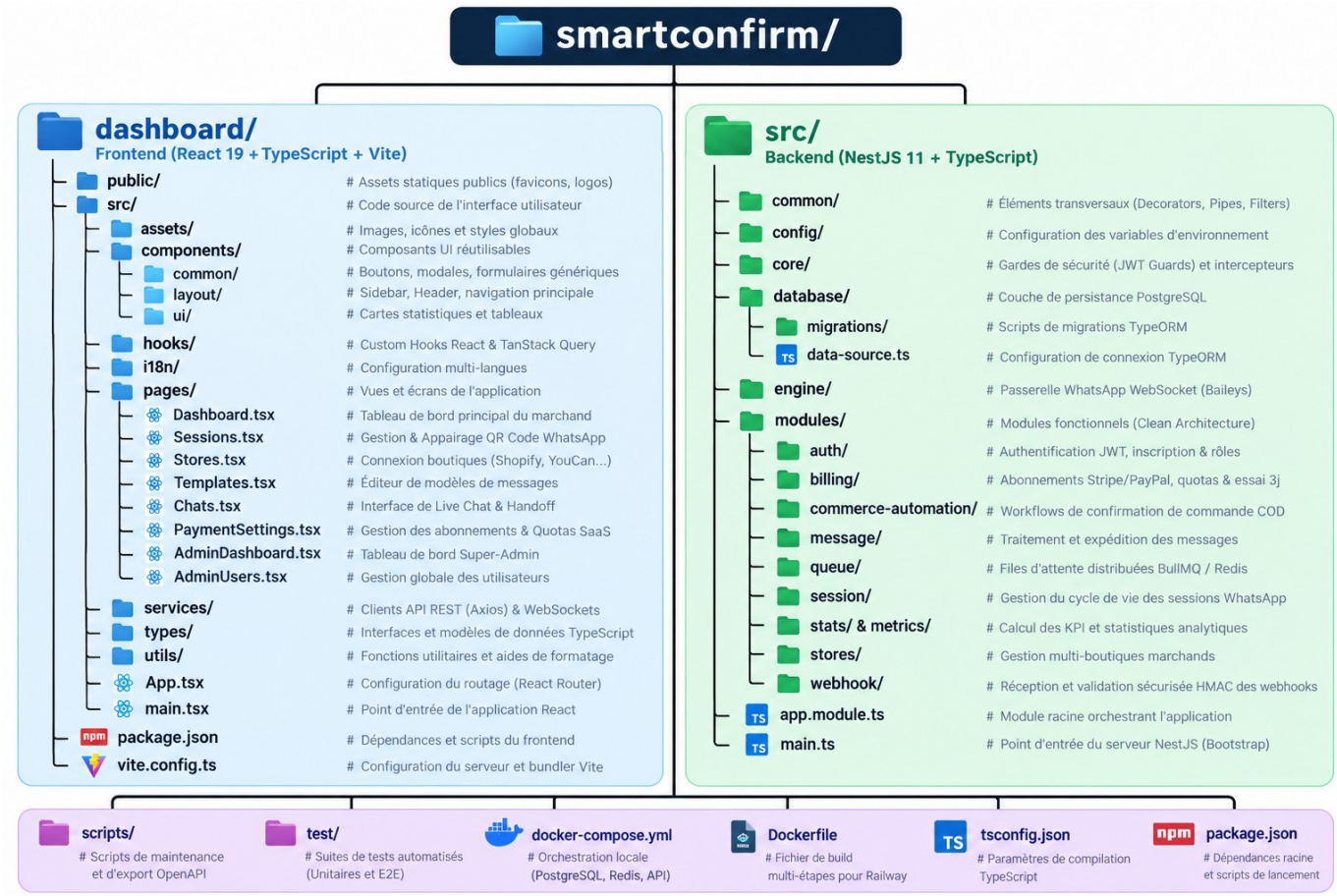
- **Vues / Interface (React SPA) :** Génèrent l'interface utilisateur dynamique et interactive pour les marchands et administrateurs, et communiquent avec l'API via des requêtes HTTP et WebSockets.
- **Contrôleurs (NestJS Controllers) :** Réceptionnent les requêtes HTTP et les Webhooks e-commerce, appliquent la sécurité (gardes JWT) et valident les données entrantes.

- **Services (Logique Métier) :** Gèrent la logique applicative (traitement des commandes, suivi des quotas, automatisation et sessions WhatsApp).

- **Modèles & Entités (TypeORM) :** Représentent les structures de données métier et assurent la communication avec la base de données PostgreSQL.

- **Files d'attente & Passerelles (BullMQ / Redis) :** Traitent les envois de messages de manière asynchrone pour réguler le débit et garantir la haute disponibilité.

Structure simplifiée :


smartconfirm/
dashboard/
Frontend (React 19 + TypeScript + Vite)
public/ # Assets statiques publics (favicons, logos)
src/ # Code source de l'interface utilisateur
assets/ # Images, icônes et styles globaux
components/ # Composants UI réutilisables
common/ # Boutons, modales, formulaires génériques
layout/ # Sidebar, Header, navigation principale
ui/ # Cartes statistiques et tableaux
hooks/ # Custom Hooks React & TanStack Query
i18n/ # Configuration multi-langues
pages/ # Vues et écrans de l'application
Dashboard.tsx # Tableau de bord principal du marchand
Sessions.tsx # Gestion & Appairage QR Code WhatsApp
Stores.tsx # Connexion boutiques (Shopify, YouCan...)
Templates.tsx # Éditeur de modèles de messages
Chats.tsx # Interface de Live Chat & Handoff
PaymentSettings.tsx # Gestion des abonnements & Quotas SaaS
AdminDashboard.tsx # Tableau de bord Super-Admin
AdminUsers.tsx # Gestion globale des utilisateurs
services/ # Clients API REST (Axios) & WebSockets
types/ # Interfaces et modèles de données TypeScript
utils/ # Fonctions utilitaires et aides de formatage
App.tsx # Configuration du routage (React Router)
main.tsx # Point d'entrée de l'application React
package.json # Dépendances et scripts du frontend
vite.config.ts # Configuration du serveur et bundler Vite
src/
Backend (NestJS 11 + TypeScript)
common/ # Éléments transversaux (Decorators, Pipes, Filters)
config/ # Configuration des variables d'environnement
core/ # Gardes de sécurité (JWT Guards) et intercepteurs
database/ # Couche de persistance PostgreSQL
migrations/ # Scripts de migrations TypeORM
data-source.ts # Configuration de connexion TypeORM
engine/ # Passerelle WhatsApp WebSocket (Baileys)
modules/ # Modules fonctionnels (Clean Architecture)
auth/ # Authentification JWT, inscription & rôles
billing/ # Abonnements Stripe/PayPal, quotas & essai 3j
commerce-automation/ # Workflows de confirmation de commande COD
message/ # Traitement et expédition des messages
queue/ # Files d'attente distribuées BullMQ / Redis
session/ # Gestion du cycle de vie des sessions WhatsApp
stats/ & metrics/ # Calcul des KPI et statistiques analytiques
stores/ # Gestion multi-boutiques marchands
webhook/ # Réception et validation sécurisée HMAC des webhooks
app.module.ts # Module racine orchestrant l'application
main.ts # Point d'entrée du serveur NestJS (Bootstrap)
scripts/
Scripts de maintenance et d'export OpenAPI
test/
Suites de tests automatisés (Unitaires et E2E)
docker-compose.yml
Orchestration locale (PostgreSQL, Redis, API)
Dockerfile
Fichier de build multi-étapes pour Railway
tsconfig.json
Paramètres de compilation TypeScript
package.json
Dépendances racine et scripts de lancement

3.3. Fonctionnalités développées

🔷 **Côté client (WhatsApp / Acheteur) :**

- Réception instantanée du récapitulatif de commande avec détails du panier et montant en paiement à la livraison (COD).
- Confirmation ou annulation de la commande en un clic via des boutons interactifs.
- Modification de l'adresse et des consignes de livraison.
- Échange avec l'assistant conversationnel IA par message texte ou note vocale (Darija, Arabe, Français).
- Évaluation du produit et dépôt d'un avis de satisfaction après réception du colis.

🔷 **Côté marchand (E-Commerçant) :**

- Authentification sécurisée (JWT), gestion du profil et des accès.
- Appairage de sessions WhatsApp en temps réel via scan de QR Code dynamique.
- Intégration et synchronisation multi-boutiques (Shopify, WooCommerce, YouCan) par Webhooks.
- Personnalisation des modèles de messages de confirmation, relance et suivi logistique.
- Interface de Live Chat avec possibilité de reprise en main manuelle des discussions (*Handoff*).
- Tableau de bord analytique pour suivre les taux de confirmation, les commandes et la délivrabilité.
- Souscription aux forfaits SaaS et consultation en direct de la consommation des quotas.

🔷 **Côté administrateur (Super-Admin) :**

- Supervision globale de la plateforme via un tableau de bord centralisé (chiffre d'affaires, marchands actifs, volume global de messages).
- Gestion des comptes utilisateurs (activation, suspension ou blocage des accès marchands).

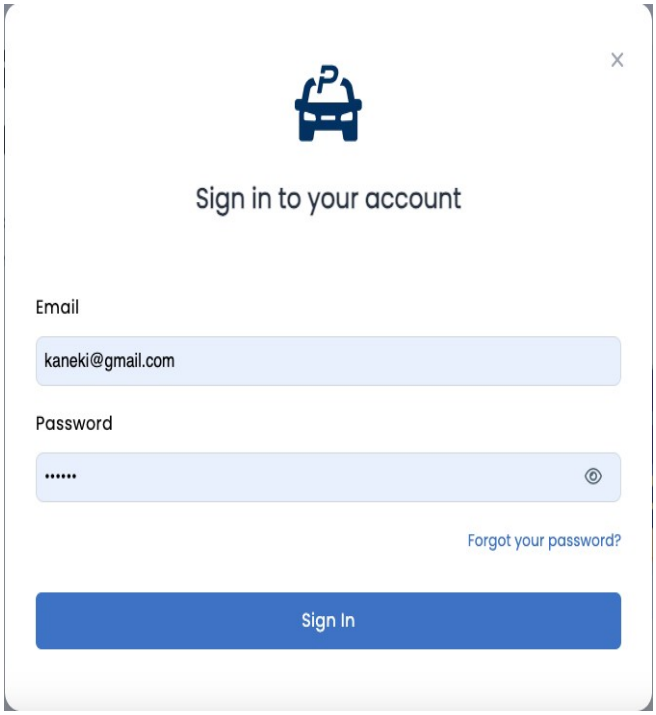
- Configuration des plans d'abonnement SaaS, des quotas autorisés et des périodes d'essai gratuites (*free trials*).
- Traitement et validation administrative des demandes de remboursement sous 48 heures.
- Surveillance de l'état de santé de l'infrastructure cloud, des conteneurs et des files d'attente BullMQ.

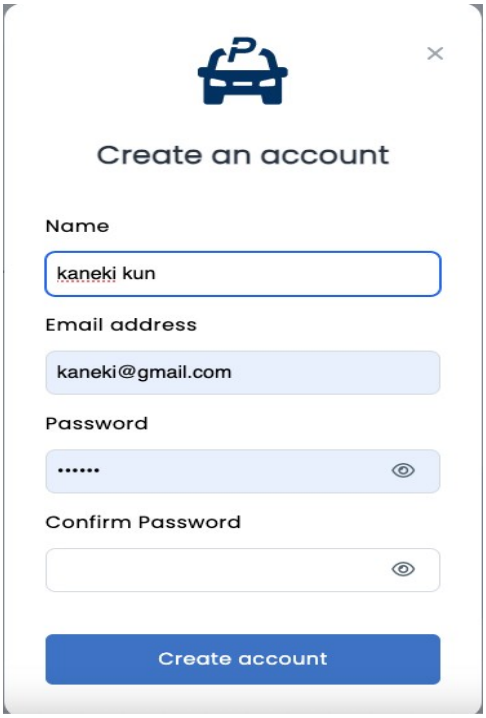
3.4. Captures d'écran de l'interface

Des captures d'écran ont été intégrées pour illustrer les principales interfaces de l'application. Ces interfaces ont été conçues de manière **responsive** pour s'adapter à tous types d'écrans.

3.5. Interfaces utilisateur – Partie publique (client)

Figure 1 : Formulaire de connexion / inscription utilisateur





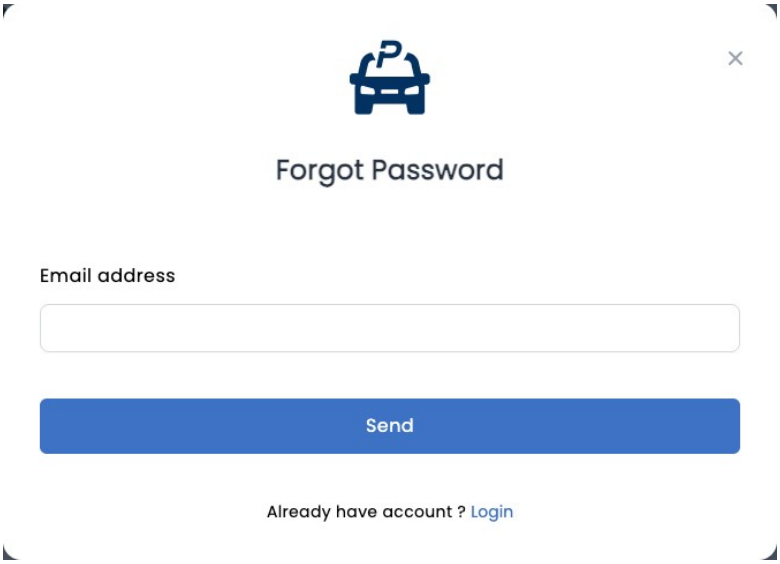
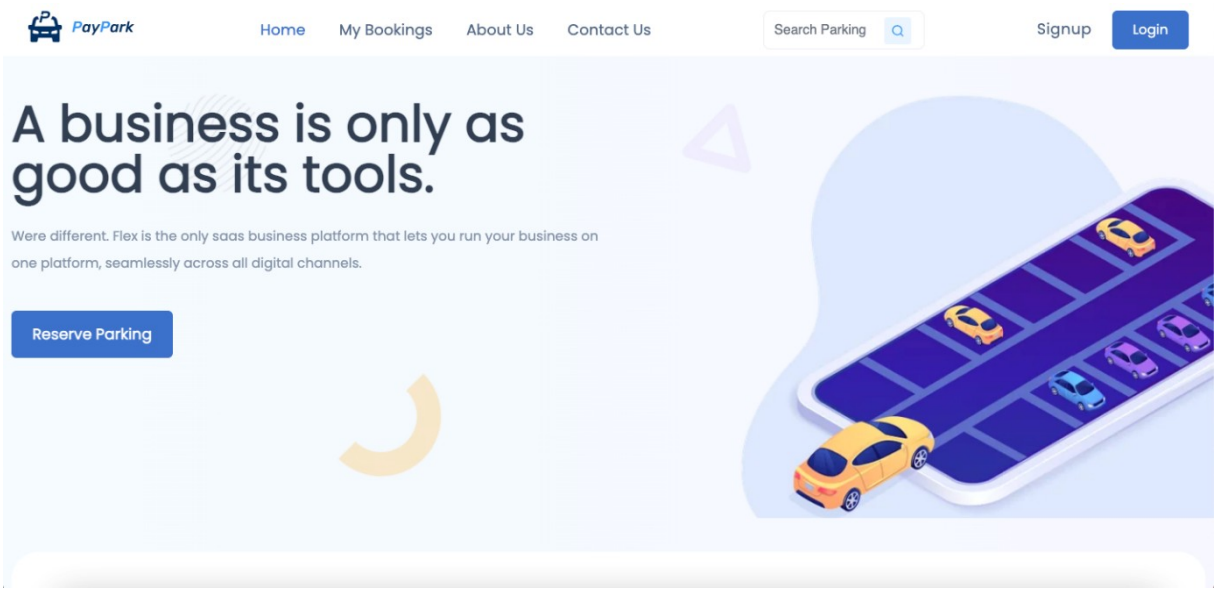
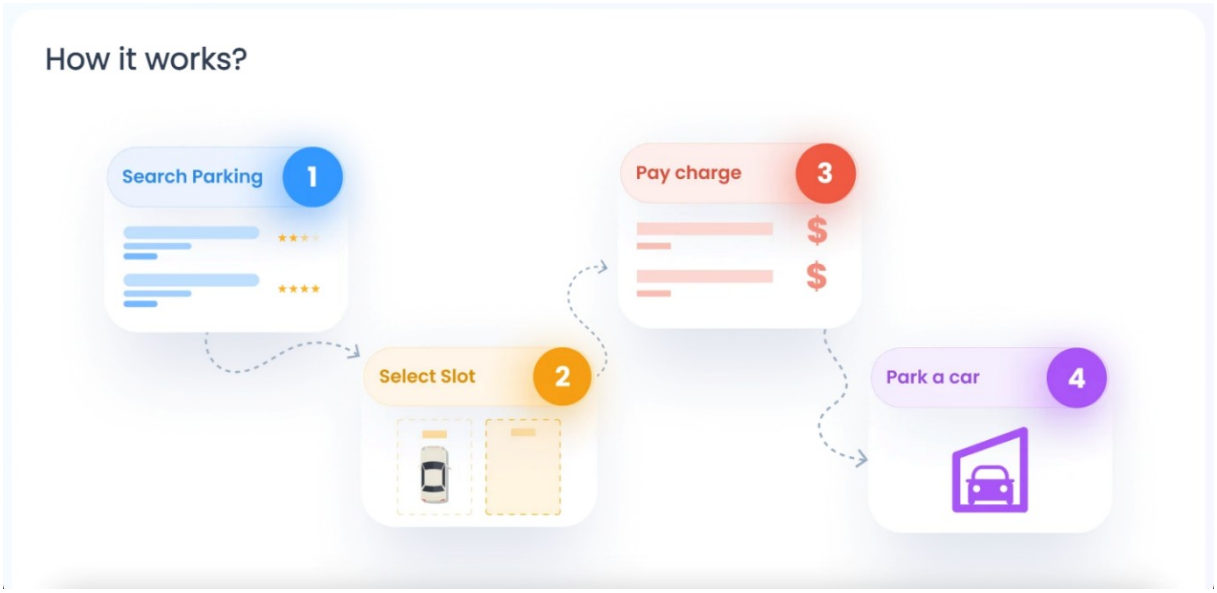


Figure 2 : Page d'accueil de l'application PayPark





Services

- 24 Hours Open
- CCTV Cameras
- EV Charging
- 24x7 Security
- Wheel-chair Accessibility
- Washing & Cleaning
- Mechanic
- Servicing

Featured Parkings



Mountain View Parking ★ 4.5

711-2880 Nulla St.Mankato
Mississippi 96522 (257) 563-7401



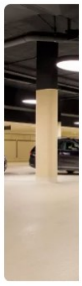
Mountain View Parking ★ 4.5

711-2880 Nulla St.Mankato
Mississippi 96522 (257) 563-7401



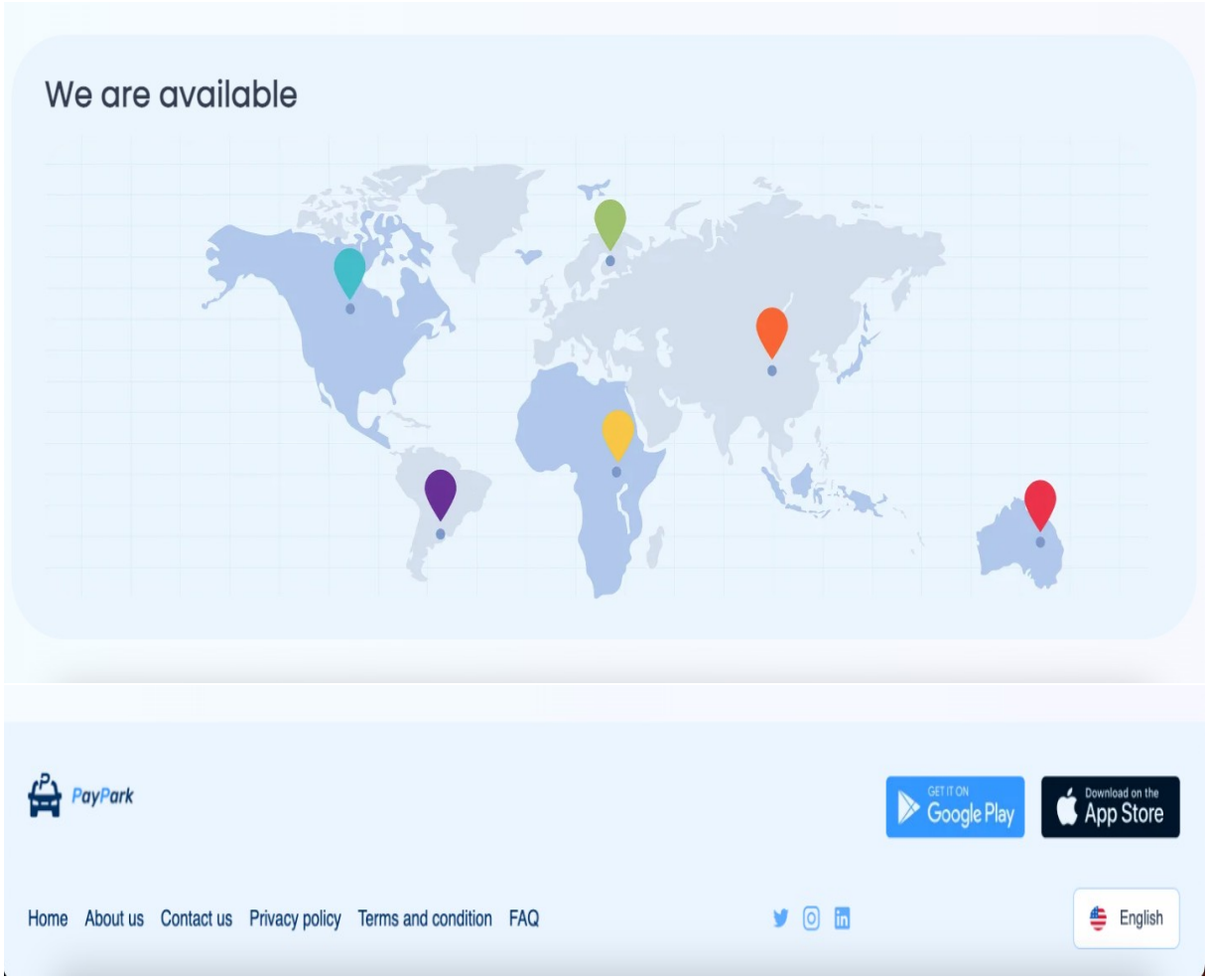
Mountain View Parking ★ 4.5

711-2880 Nulla St.Mankato
Mississippi 96522 (257) 563-7401



Mountair

711-288
Mississi


We are available
PayPark
GET IT ON Google Play
Download on the App Store
Home
About us
Contact us
Privacy policy
Terms and condition
FAQ
English

c’est important premièrement de l’ajouter votre véhicule tout d’abord avant de réserver un parking

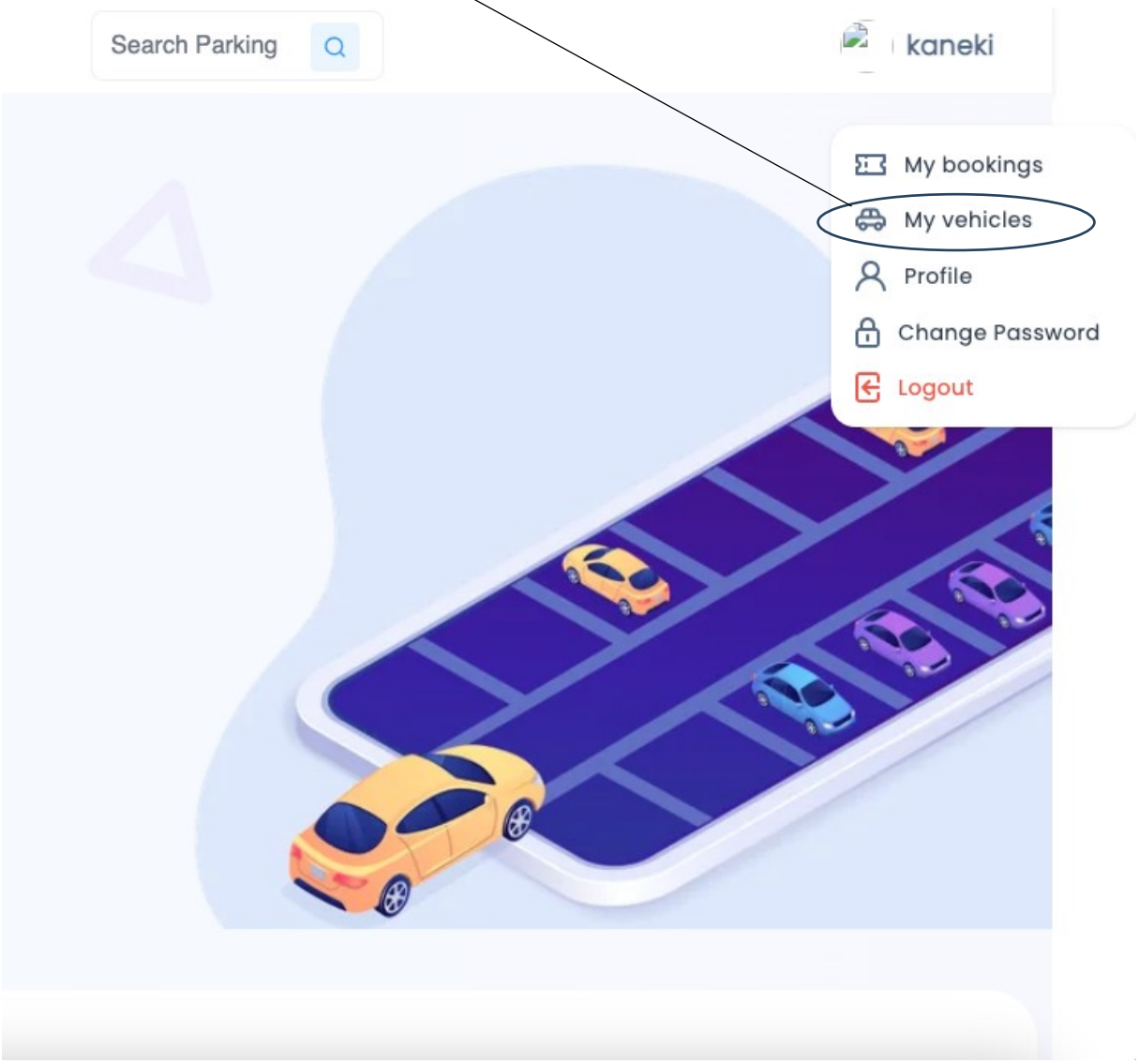


Figure 3 : Formulaire d'ajout d'un véhicule

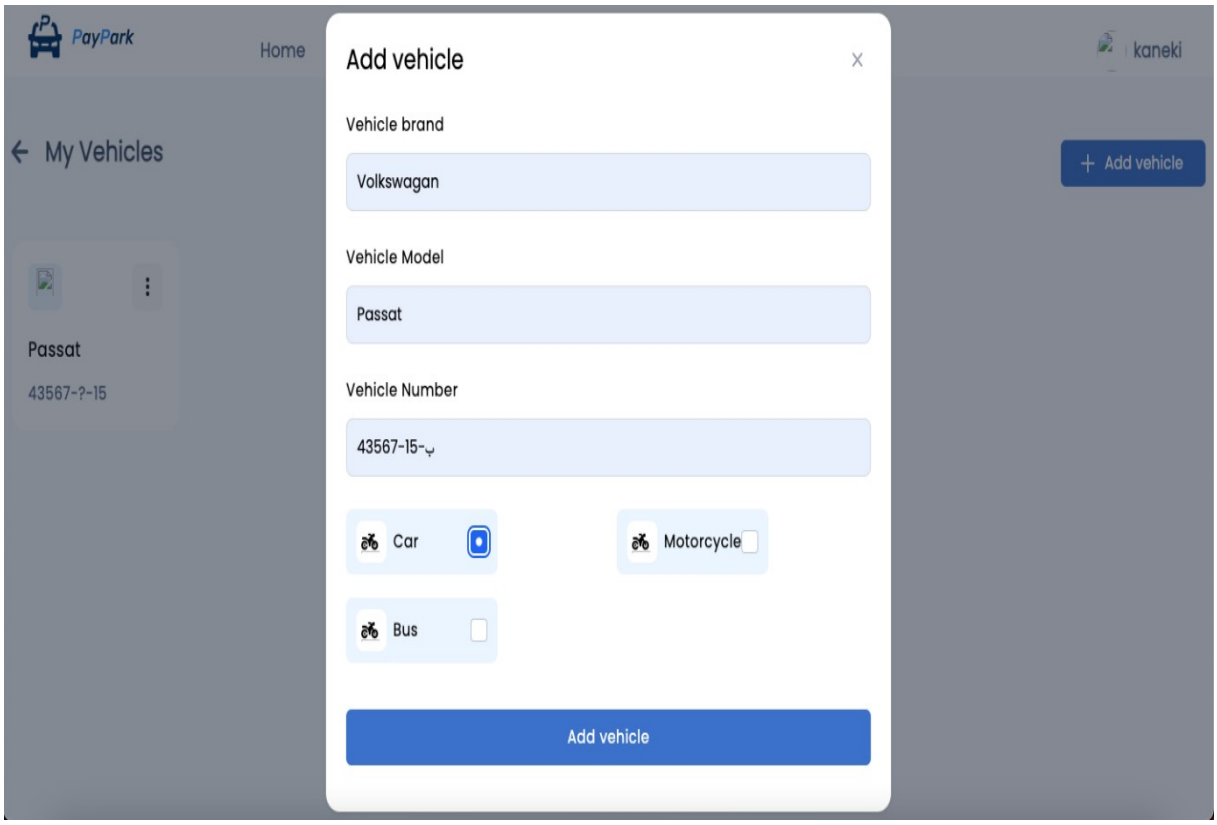


Figure 4 : Interface de recherche de parking par localisation

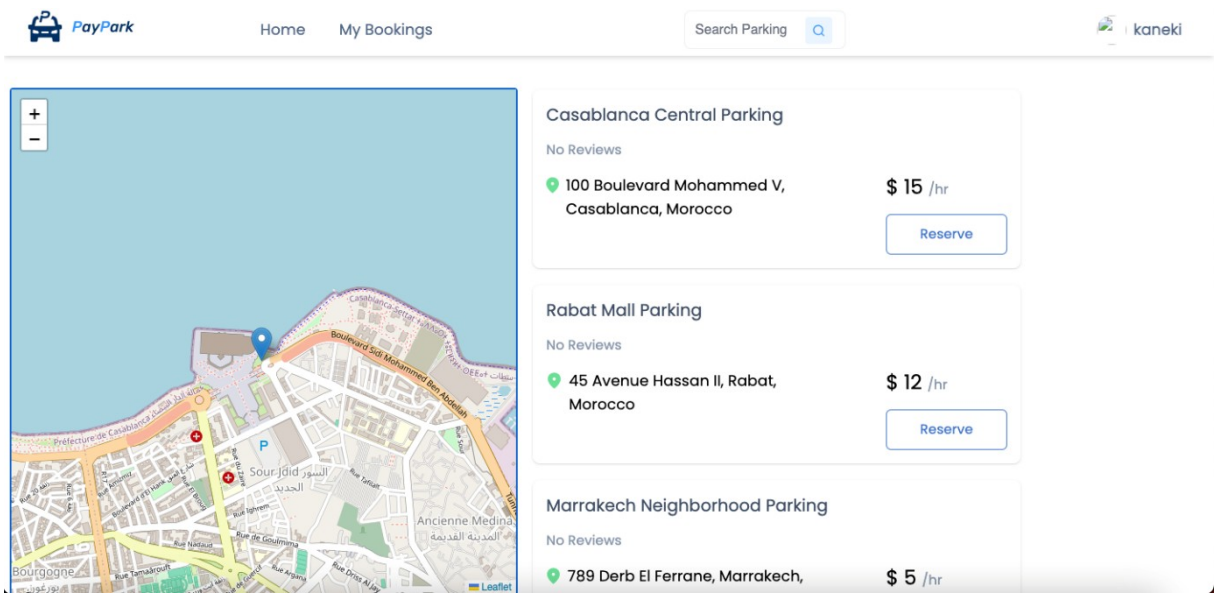


Figure 5 : Page de réservation – choix de date et heure

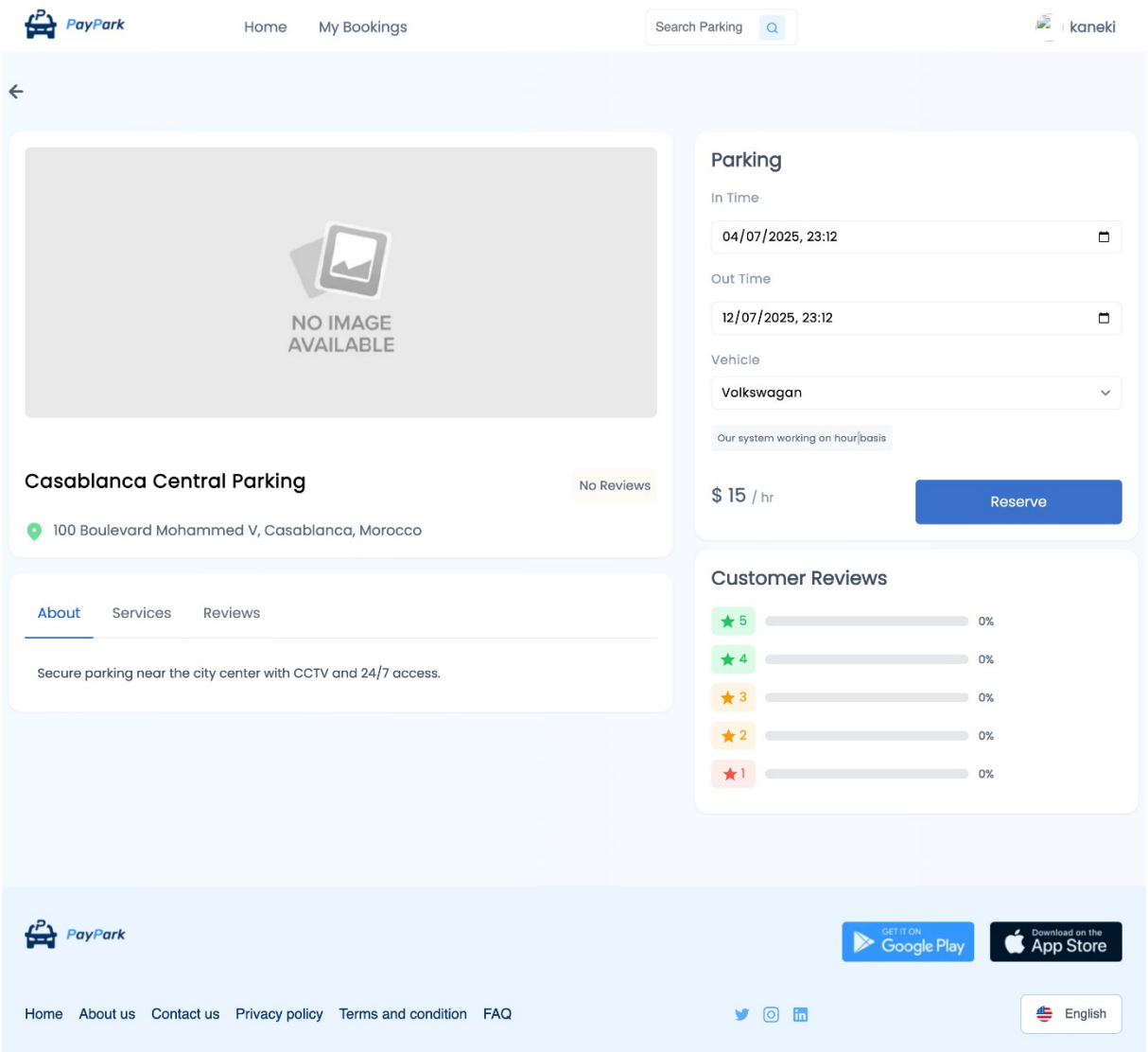


Figure 6 : Sélection d'une place de parking selon disponibilité

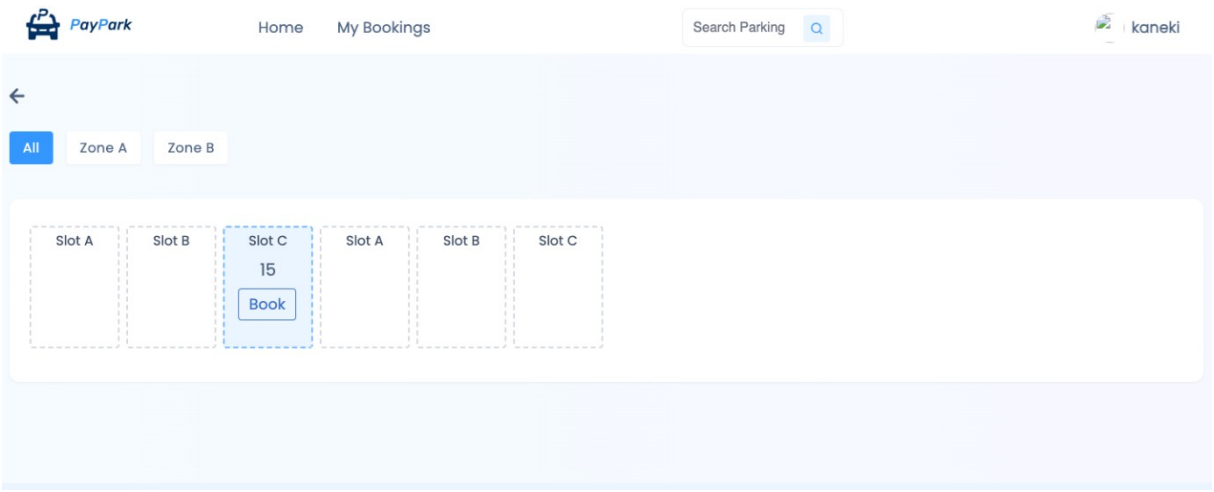


Figure 7 : Interface de paiement en ligne (Stripe / PayPal – mode test)

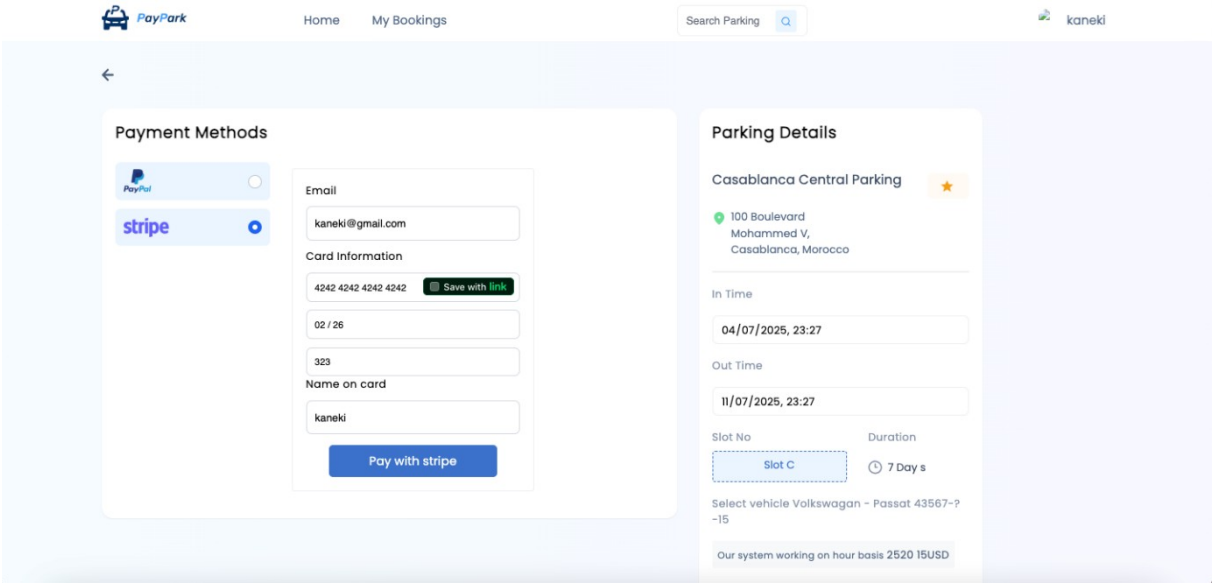


Figure 8 : Confirmation de réservation par la propriétaire

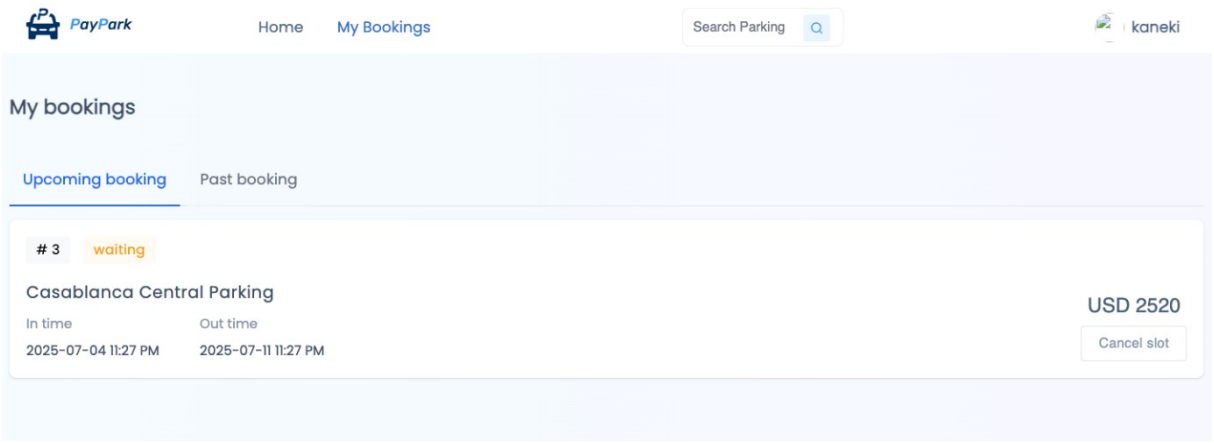
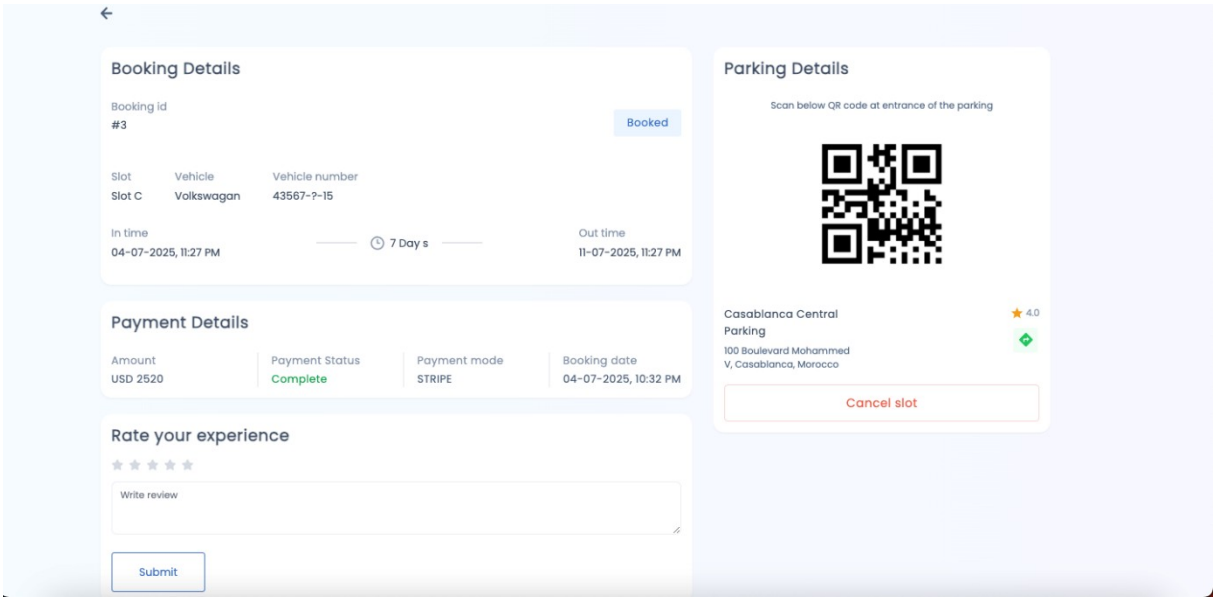


Figure 9 : Confirmation de réservation avec QR code



3.6. Interfaces utilisateur – Partie propriétaire (owner)

Figure 10 : Tableau de bord du propriétaire

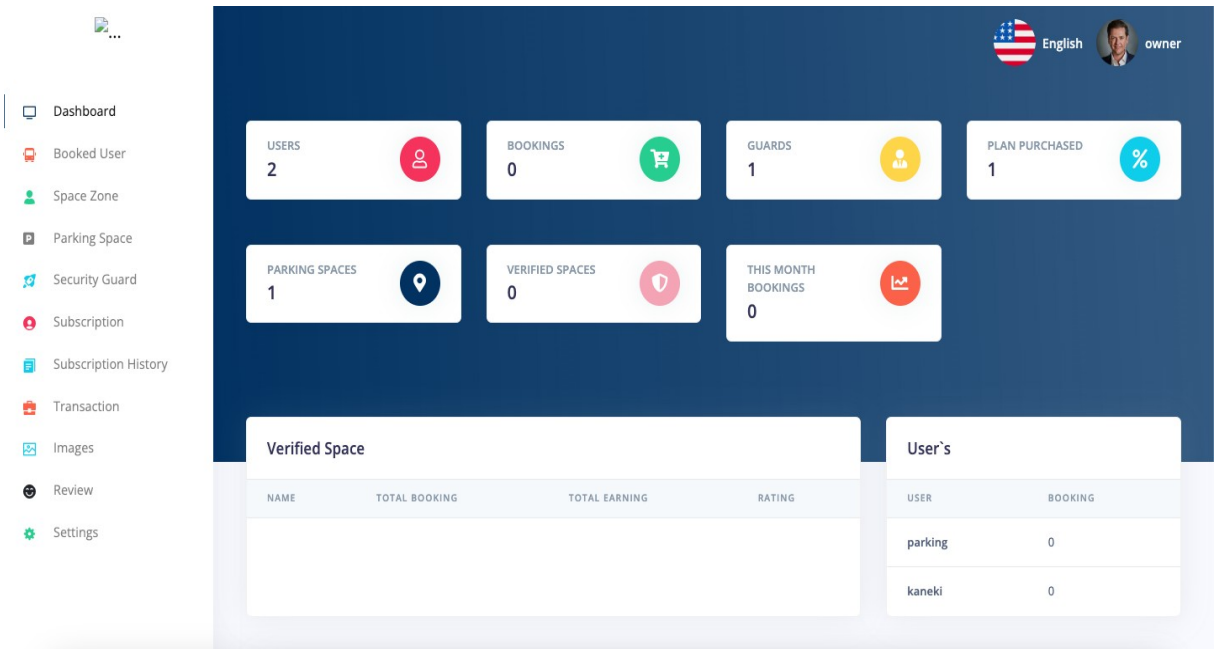
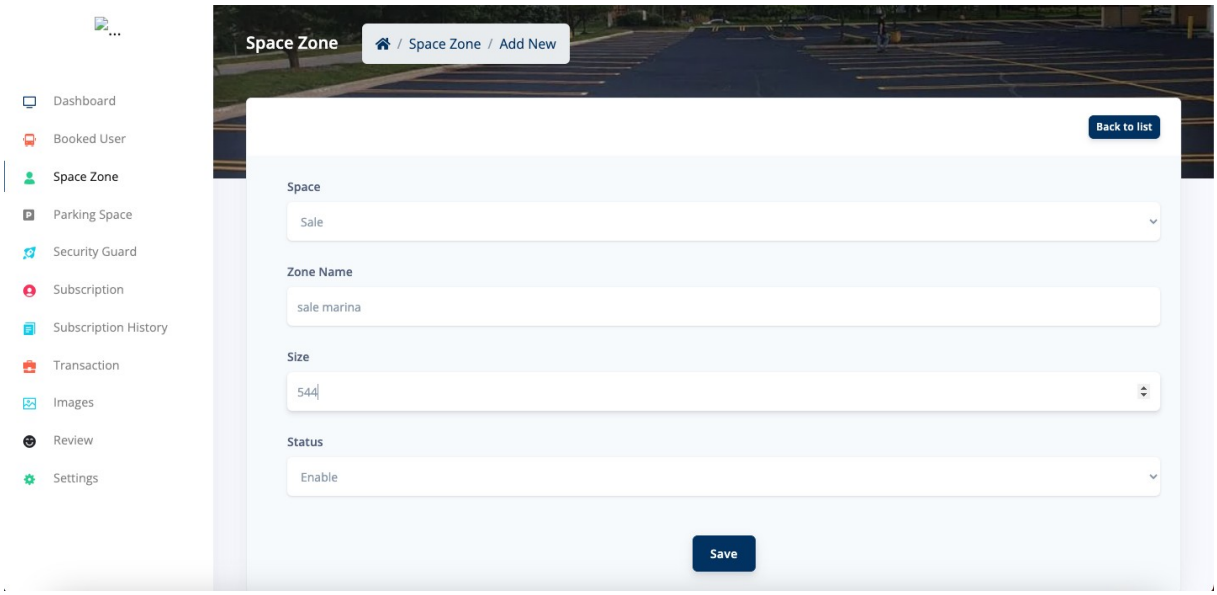


Figure 11 : Formulaire d'ajout d'un nouveau parking



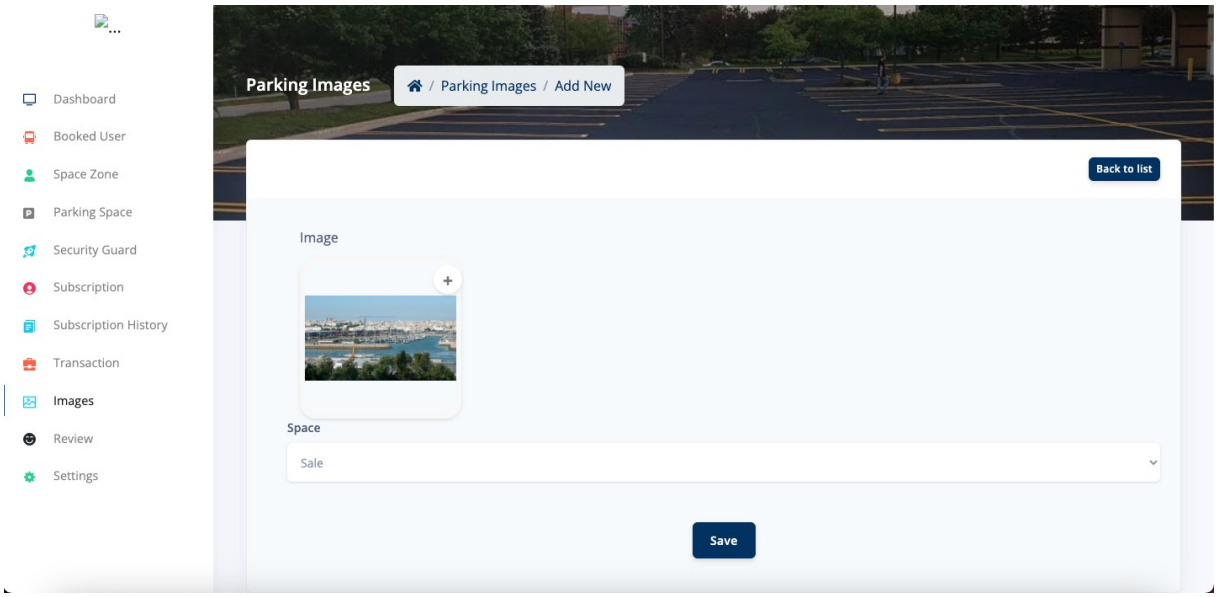


Figure 12 : Formulaire d'ajout d'un nouveau gardien

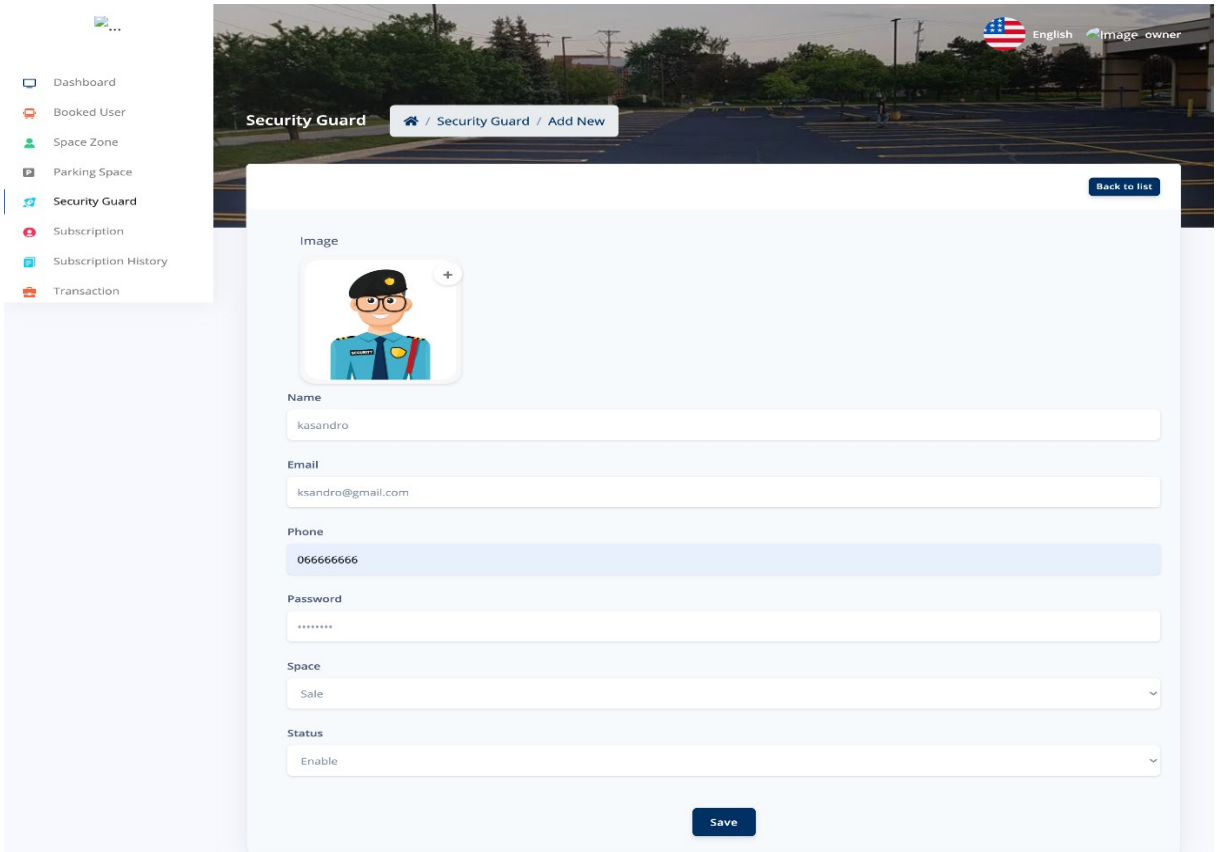
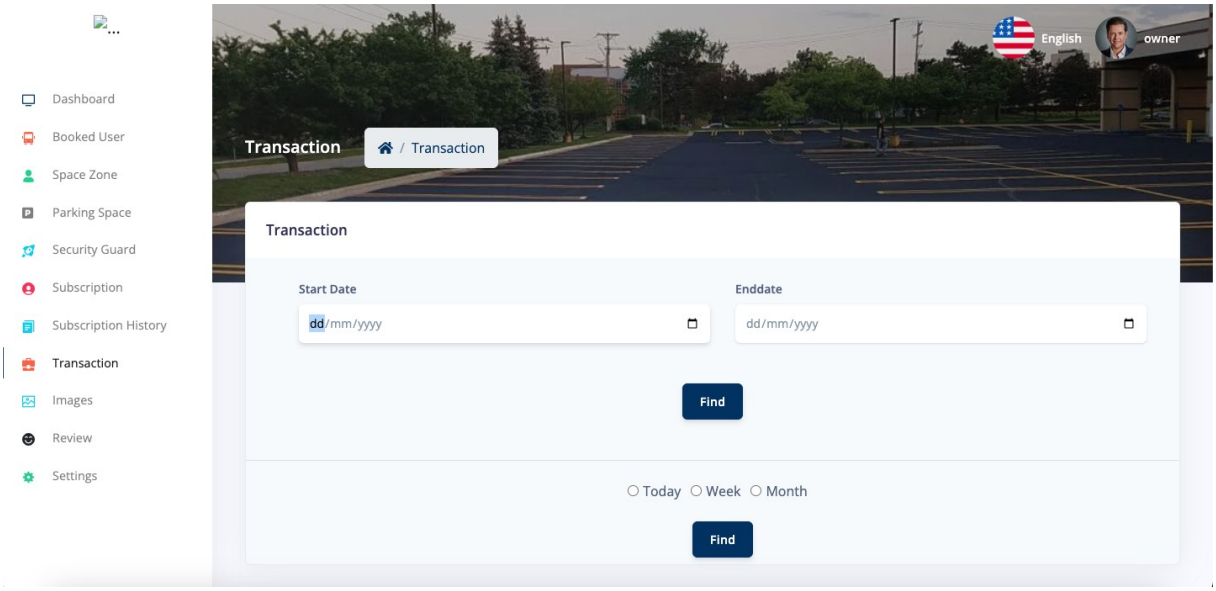


Figure 13 : Interface de suivi des transactions effectués



3.7. Interfaces utilisateur – Partie administrateur (admin)

Figure 14 : Tableau de bord administrateur

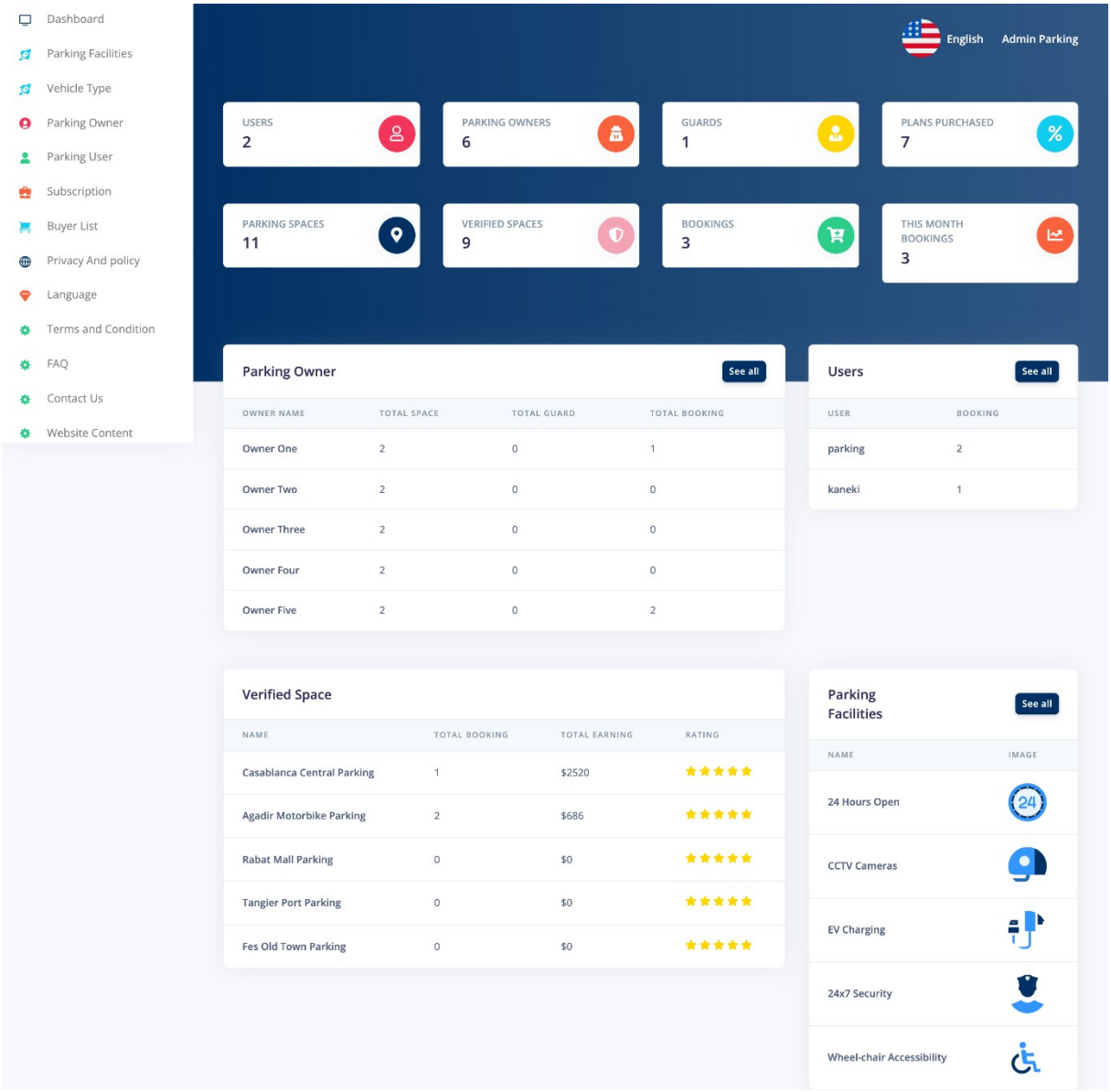


Figure 15 : Liste des parkings et leurs facilités

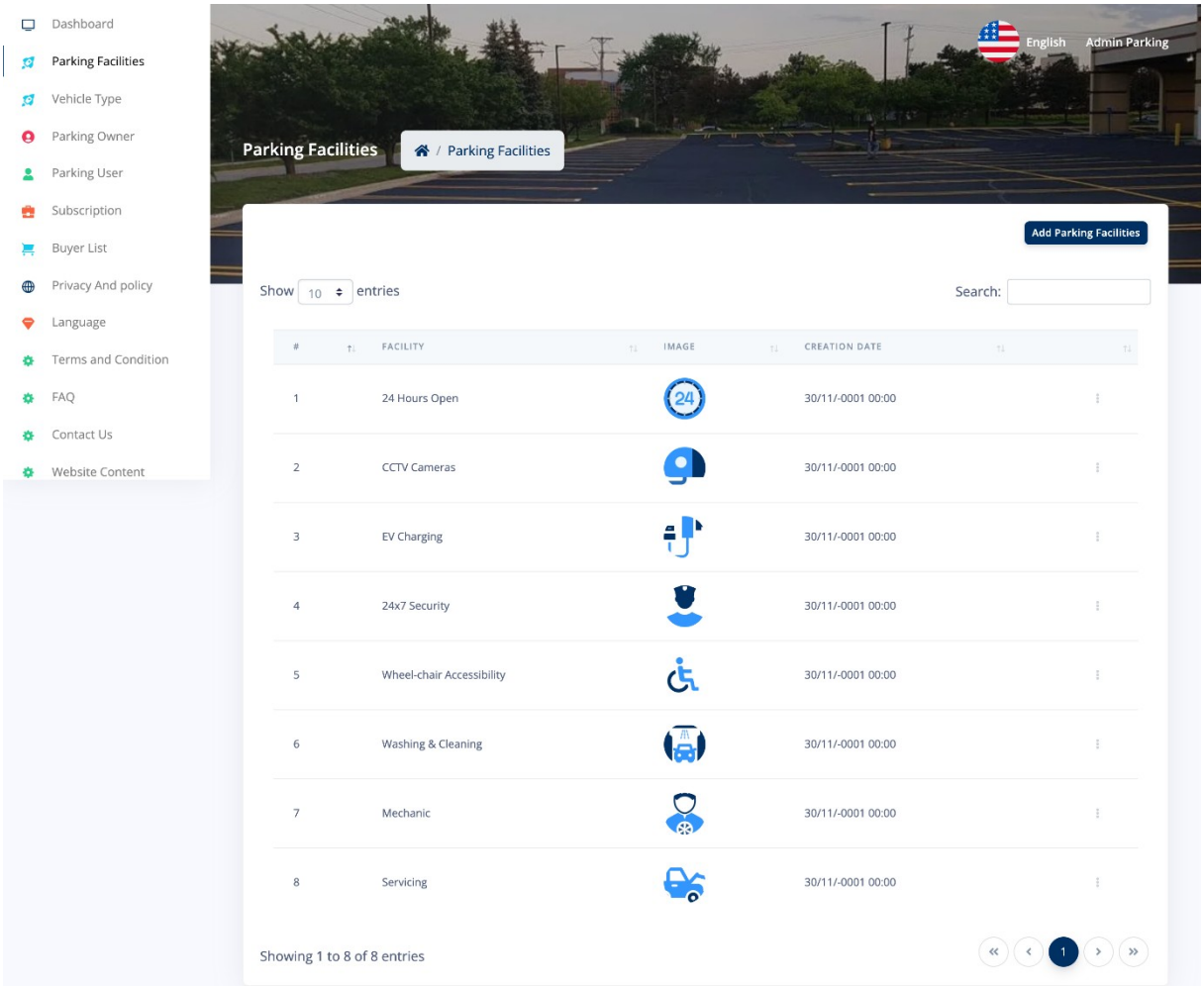


Figure 16 : Interface de gestion des types de véhicules

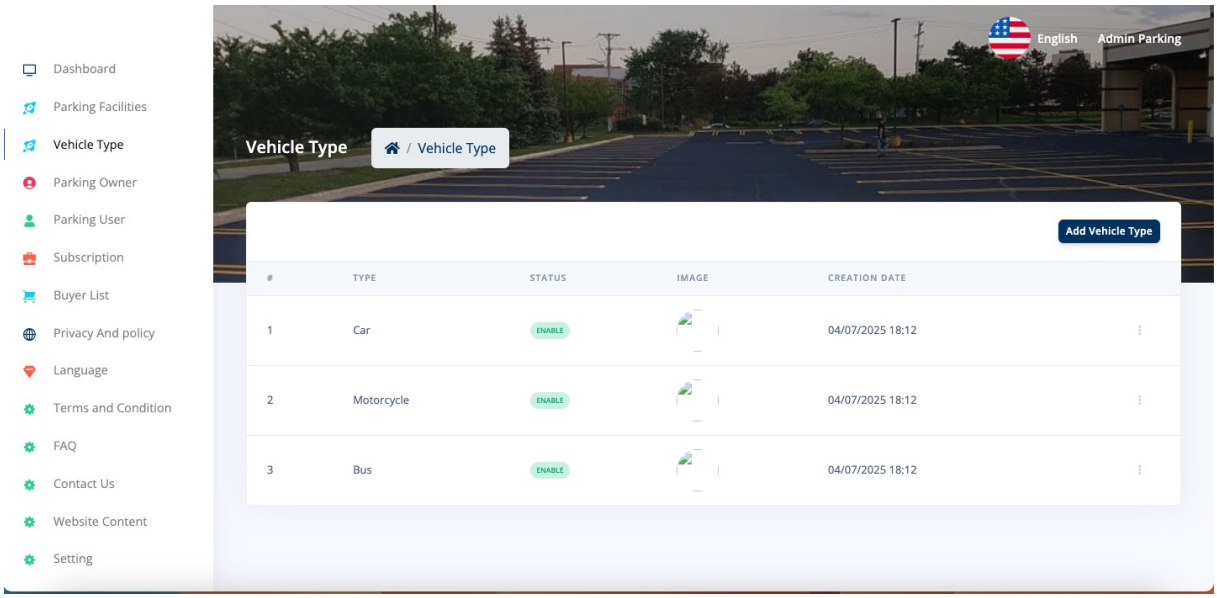


Figure 17 : Interface de gestion des propriétaires

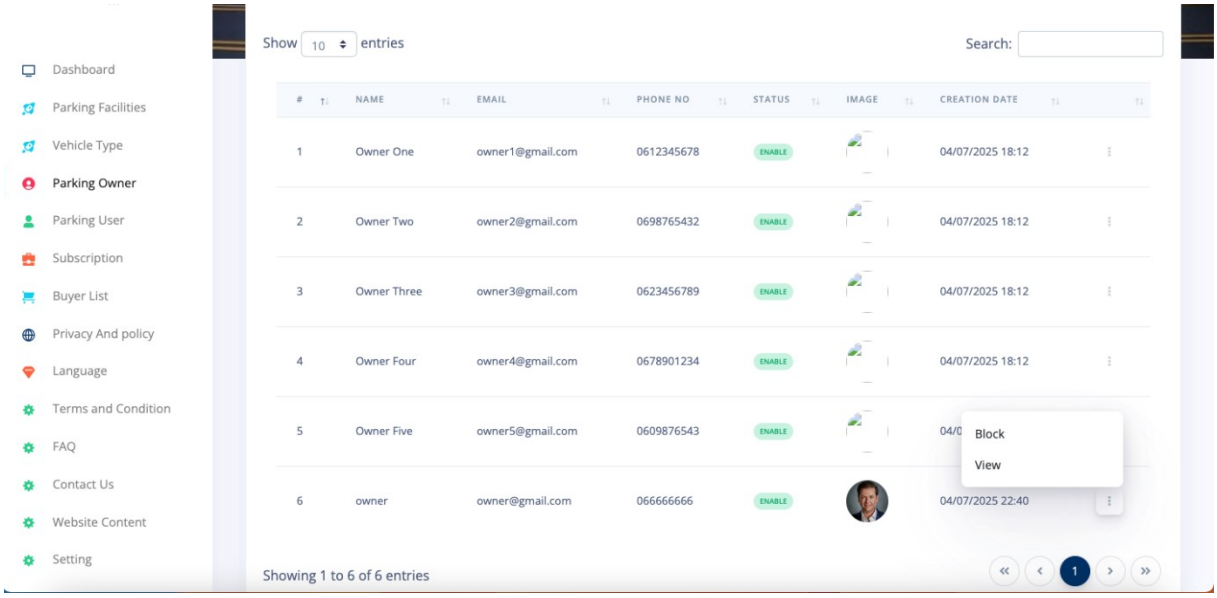
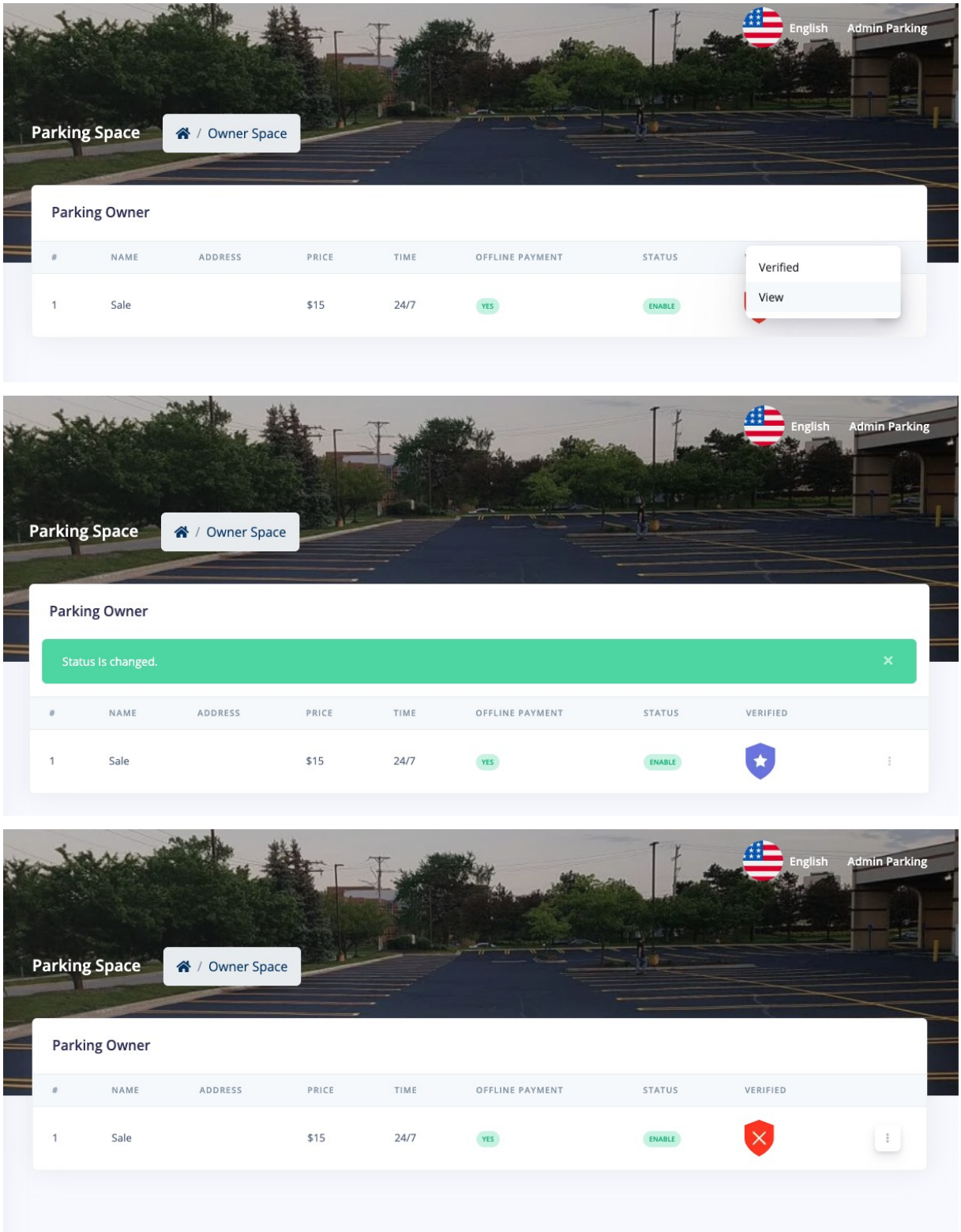


Figure 18 : Interface de vérification des parkings des propriétaires



Chapitre 4 : Tests et Déploiement

4.1. Stratégie de test

Pour assurer la fiabilité et le bon fonctionnement de l'application **SmartConfirm**, plusieurs types de tests ont été réalisés tout au long du développement :

Tests fonctionnels :

Ils ont permis de vérifier que chaque fonctionnalité développée répond correctement aux besoins exprimés :

- Inscription, connexion sécurisée et gestion des profils marchands.
- Appairage des sessions WhatsApp par scan en direct de QR Code (WebSocket / Baileys).
- Réception des Webhooks et synchronisation des commandes e-commerce (Shopify, WooCommerce, YouCan).
- Envoi automatisé des messages de confirmation et traitement interactif des réponses clients.
- Souscription aux forfaits SaaS, gestion de la période d'essai de 3 jours (*trial*) et contrôle des quotas.

Tests d'interface :

Des tests manuels ont été effectués pour s'assurer que :

- L'interface s'affiche de manière fluide et conforme sur les différents navigateurs (Chrome, Firefox, Edge, Safari).

- L'affichage est parfaitement adaptatif (*responsive*) sur ordinateur, tablette et smartphone grâce à Tailwind CSS.

- Les événements temps réel réagissent instantanément sans rechargement de page (apparition du QR Code, statut des messages).

- Les formulaires intègrent des états de chargement clairs, des retours visuels immédiats et des notifications toast explicites.

Tests de sécurité (de base) :

- Protection des routes de l'API NestJS par des gardes d'authentification par tokens JWT (*Guards*).

- Contrôle strict du cloisonnement des rôles et des autorisations (*Super-Admin* vs *Marchand*).

- Hachage cryptographique robuste des mots de passe avec **bcrypt** et chiffrement des sessions WhatsApp.

- Protection contre les attaques par déni de service grâce au limiteur de débit (*Rate Limiting / Throttler*) et aux en-têtes sécurisés (**Helmet**).

- Validation des signatures cryptographiques HMAC sur les Webhooks entrants pour bloquer les requêtes frauduleuses.

Tests de validation :

- Contrôle strict des champs obligatoires et des formats de données entrants via les DTO NestJS et class-validator (adresses email, numéros de téléphone au format international E.164).

- Contrôle de conformité des plans tarifaires et vérification du solde de messages disponibles avant tout envoi.

- Renvoi de messages d'erreurs clairs et compréhensibles en cas de saisie invalide.

4.2. Résultats des tests

Les tests effectués ont permis de confirmer que les principales fonctionnalités de l'application fonctionnent correctement :

FONCTIONNALITÉ TESTÉE	RÉSULTAT ATTENDU	RÉSULTAT OBTENU
INSCRIPTION & AUTHENTIFICATION MARCHAND	Création du compte, génération du token JWT et redirection vers le dashboard	OK
APPAIRAGE D'UNE SESSION WHATSAPP	Affichage du QR Code en direct via WebSockets et connexion établie avec succès	OK
RÉCEPTION WEBHOOK D'UNE COMMANDE	Détection automatique de la commande e-commerce et insertion en base de données	OK
ENVOI DU MESSAGE DE CONFIRMATION WHATSAPP	Message reçu instantanément sur le téléphone du client avec boutons d'action	OK
VALIDATION PAR LE CLIENT (CLIC OU AUDIO)	Mise à jour du statut en « Confirmé » et notification temps réel sur le tableau de bord	OK
CONTRÔLE D'ACCÈS NON AUTORISÉ	Tentative d'accès marchand aux routes Super-Admin bloquée avec une erreur 403 (*Forbidden*)	OK
DÉPASSEMENT DU QUOTA DE MESSAGES	Blocage automatique de l'envoi et alerte invitant à faire évoluer le forfait SaaS	OK
DEMANDE DE REMBOURSEMENT (< 48H)	Enregistrement de la requête et notification dans l'espace administrateur	OK

En cas de bug ou de dysfonctionnement mineur (notamment la gestion des reconnexions WebSocket ou la temporisation des files d'attente BullMQ), des ajustements et correctifs ont été intégrés directement dans le code source pour assurer une stabilité maximale de la plateforme.

4.3. Déploiement de l'application

Environnement de développement local :

Le projet a d'abord été développé et testé localement à l'aide d'outils modernes, permettant l'utilisation de :

- **Node.js (v22+) & NestJS CLI** pour l'exécution et le rechargement du serveur d'API en local.
- **Vite & React 19** pour le serveur de développement frontend avec rechargement à chaud (*HMR*).
- **Docker & Docker Compose** pour exécuter de façon isolée la base de données **PostgreSQL** et le cache **Redis**.
- **DBeaver / pgAdmin** pour l'administration graphique et la visualisation des tables PostgreSQL.

Déploiement en ligne :

Le projet a ensuite été déployé en production sur la plateforme cloud **Railway** (PaaS), avec :

- **Déploiement continu (CI/CD) automatisé à chaque mise à jour poussée sur le dépôt GitHub.**
- **Construction d'un conteneur optimisé grâce à un Dockerfile multi-étapes (*multi-stage build*) empaquetant le backend NestJS et le build de production React.**
- **Provisionnement des services managés pour la base de données Railway PostgreSQL et le cache Railway Redis.**
- **Configuration centralisée des variables d'environnement (.env) directement sur l'interface de Railway.**
- **Sécurisation par certificat SSL/TLS automatique assurant la protection de toutes les routes et des Webhooks en HTTPS.**

Lien du projet en ligne : https://pfa-production-7b71.up.railway.app/

Outils utilisés pour le déploiement :

- **GitHub :** pour le versionnement du code source et le déclenchement automatique des builds de déploiement.
- **Docker :** pour la conteneurisation de l'application et la garantie d'un environnement d'exécution reproductible.
- **Railway :** comme plateforme cloud d'hébergement (PaaS), gérant l'orchestration des conteneurs, le routage réseau et les certificats SSL.
- **Console Railway :** pour la surveillance en temps réel des logs serveur, des métriques d'utilisation (RAM/CPU) et des bases de données

Chapitre 5 :

Difficultés rencontrées et perspectives d'évolution

5.1. Difficultés rencontrées

Durant la réalisation du projet **SmartConfirm**, plusieurs obstacles ont été rencontrés, tant sur le plan technique que méthodologique :

Gestion du protocole WhatsApp & persistance des sessions :

- L'intégration de la bibliothèque WebSocket **Baileys** a demandé un apprentissage approfondi du protocole interne de WhatsApp Web, notamment pour capturer les flux d'événements, générer les QR Codes dynamiques et gérer la reconnexion automatique en cas de coupure réseau.

- La mise en place de politiques de régulation de débit (*rate limiting* et délais aléatoires via **BullMQ / Redis**) a été indispensable pour éviter le risque de détection et de bannissement des numéros par WhatsApp lors d'envois simultanés.

Synchronisation temps réel et architecture Full-Stack :

- La mise en place d'un système de **gestion de rôles (admin, propriétaire, client)** a demandé une bonne organisation des routes, des middlewares et des vues spécifiques à chaque type d'utilisateur.

Intégration hétérogène des plateformes e-commerce :

- La normalisation des données issues de CMS différents (**Shopify, WooCommerce, YouCan**) a représenté un défi, chaque plateforme ayant son propre format de Webhook et ses règles de vérification de signature cryptographique (HMAC).

Déploiement et conteneurisation cloud :

- La conception d'un Dockerfile multi-étapes (*multi-stage build*) optimisé, capable de compiler à la fois le backend NestJS et le frontend React tout en réduisant l'empreinte mémoire pour la plateforme cloud **Railway**.

- La configuration et la liaison des services managés **PostgreSQL** et **Redis** avec gestion sécurisée des variables d'environnement en production.

5.2. Limites du projet

Bien que la plateforme **SmartConfirm** intègre un moteur de facturation et d'abonnements complet, celui-ci fonctionne pour le moment en mode test (*Sandbox*) à travers les passerelles **Stripe** et **PayPal**.

Ces deux solutions ont été privilégiées pour leur conformité internationale et leur robustesse dans la gestion des prélèvements récurrents et des périodes d'essai de 3 jours.

Limites restantes :

- Les paiements en ligne ne sont pas encore basculés en mode production avec des comptes bancaires marchands réels.
- La reconnaissance vocale en dialecte local (Darija) via les modèles d'IA dépend de la qualité acoustique de l'enregistrement et nécessite parfois une intervention humaine (*Live Chat / Handoff*).
- Dépendance vis-à-vis des évolutions du protocole WhatsApp Web, nécessitant un suivi régulier des mises à jour de la bibliothèque Baileys.
- L'accès au tableau de bord s'effectue pour le moment exclusivement via navigateur web (bien qu'ergonomique et responsive), en l'absence d'une application mobile native dédiée.

5.3. Perspectives d'évolution

Monétisation et exploitation commerciale :

- Finaliser le passage en production des passerelles Stripe et PayPal pour encaisser des abonnements réels.
- Enrichir le tableau de bord Super-Admin avec des métriques financières avancées (MRR - *Monthly Recurring Revenue*, Churn rate, Lifetime Value).

Intelligence Artificielle & automatisation avancée :

- Entraîner un modèle LLM affiné (*fine-tuned*) spécifiquement pour la négociation commerciale en Darija marocaine (ex. proposer une réduction automatique si le client hésite à confirmer).
- Intégrer un module de synthèse vocale neuronale (TTS) permettant au bot de répondre directement aux clients par des notes vocales personnalisées.

Améliorations techniques & Écosystème :

- Rendre la plateforme multi-canale en étendant le moteur de confirmation à d'autres réseaux sociaux (**Instagram Direct Messaging**, **Messenger**, **TikTok Shop**).

- Développer une application mobile native (via **React Native** ou **Flutter**) permettant aux e-commerçants de gérer leurs relances et leur Live Chat directement depuis leur smartphone.

- Connecter le système aux API des transporteurs et sociétés de livraison locales pour générer automatiquement les bordereaux d'expédition dès la validation du client sur WhatsApp.

Améliorations techniques :

- Rendre le système **multilingue (AR/EN)**.
- Ajouter un **système de notation et de commentaires** pour les utilisateurs.
- Créer une **version mobile** dédiée via Flutter ou React Native connectée à une API Laravel.

Conclusion Générale

Le projet **SmartConfirm** a été une expérience particulièrement enrichissante sur le plan technique, organisationnel et personnel. À travers ce travail, j'ai pu mettre en pratique l'ensemble des compétences acquises durant ma formation d'Ingénieur en Génie Informatique à l'**ISGA**, notamment en modélisation UML, en développement Full-Stack moderne avec **React 19** et **NestJS (TypeScript)**, en gestion de bases de données relationnelles avec **PostgreSQL** et **TypeORM**, en traitement asynchrone des flux via **Redis/BullMQ**, ainsi qu'en conteneurisation et déploiement continu sur la plateforme cloud **Railway**.

L'objectif principal du projet était de moderniser et d'automatiser le processus de confirmation des commandes e-commerce (notamment en paiement à la livraison - COD) à travers une plateforme SaaS centralisée, réactive et intuitive. Grâce aux fonctionnalités adaptées à chaque profil (e-commerçant, super-administrateur et client final WhatsApp), le système répond concrètement aux défis logistiques des vendeurs en ligne en sécurisant les expéditions et en diminuant les taux de retour de colis. La solution développée est pleinement fonctionnelle, stable, sécurisée et évolutive.

Ce projet constitue une base solide pour des perspectives d'évolution futures, telles que l'intégration de modèles d'IA conversationnelle plus poussés pour la négociation commerciale, la connexion avec d'autres canaux de messagerie (Instagram Direct, Messenger), ou encore la conception d'une application mobile dédiée aux livreurs. Il a considérablement renforcé ma capacité à concevoir et piloter un projet informatique complexe de bout en bout, à résoudre des problématiques d'intégration réelles et à produire une solution SaaS directement exploitable dans l'écosystème e-commerce.

Bibliographie / Webographie

Bibliographie

1. **Jean-François Bérard**, *Conception et développement d'applications web*, Éditions ENI, 2020.
2. **Bertrand Keller**, *UML 2 – Pratique de la modélisation*, Éditions Eyrolles, 2019.
3. **Sébastien Castiel**, *TypeScript et React – Concevoir des applications web modulaires et typées*, Éditions Eyrolles, 2023.
4. **Marc Pomar & Greg Gorlen**, *NestJS Progressive Framework – Architectures d'entreprise et microservices Node.js*, Packt Publishing, 2022.
5. **Bertrand Estellon**, *Bases de données relationnelles – SQL, conception et optimisation*, Dunod, 2020.
6. **Sébastien Lardière**, *PostgreSQL – Administration, exploitation et haute disponibilité*, Éditions ENI, 2021

Webographie

1. **NestJS Documentation officielle** – https://docs.nestjs.com
2. **React 19 Documentation officielle** – https://react.dev
3. **TypeScript** – https://www.typescriptlang.org/docs
4. **Baileys (Passerelle WhatsApp WebSocket)** – https://github.com/WhiskeySockets/Baileys
5. **PostgreSQL Documentation officielle** – https://www.postgresql.org/docs
6. **TypeORM Guides & Documentation** – https://typeorm.io
7. **BullMQ & Redis (Traitement asynchrone des files d'attente)** – https://docs.bullmq.io
8. **Stripe Developers Portal (Abonnements & Billing)** – https://stripe.com/docs
9. **PayPal Developer Documentation** – https://developer.paypal.com
10. **Railway Documentation (Hébergement Cloud PaaS & CI/CD)** – https://docs.railway.app
11. **Docker Documentation officielle** – https://docs.docker.com
12. **Tailwind CSS Documentation** – https://tailwindcss.com/docs

13. **Git & GitHub Documentation** – https://docs.github.com

14. **diagrams.net (draw.io - Outil de modélisation UML)** – https://app.diagrams.net